



RISC-V External Debug Security Specification

Version v0.7.6, 2026-09-09: Updated Release Candidate for ARC Review

Table of Contents

Preamble	1
Copyright and license information	3
Contributors	5
1. Introduction	7
1.1. Control Hierarchy	7
1.1.1. External Debug Security Control	8
1.1.2. Trace Security Control	9
1.2. Terminology	10
2. External Debug Security Threat Model	11
3. Secure Debug and Secure Trace (ISA Extensions)	13
3.1. External Debug Security Extensions	13
3.1.1. External Debug Security Control States and CSRs	13
3.1.2. Debug Operations	15
3.1.3. Debug Access Privilege	15
3.1.4. Allowed Resume Privilege Modes	16
3.1.5. M-mode External Debug Security Extension (Smmedbgsec)	16
3.1.6. S/HS-mode External Debug Security Extension (Smsedbgsec)	17
sdcsr and sdpc	18
Debug Access Privilege for Memory in Smsedbgsec	19
3.1.7. VS-mode External Debug Security Extension (Smvsedbgsec)	20
sdcsr and sdpc in VS-mode	20
Debug Access Privilege for Memory in Smvsedbgsec	21
3.1.8. U-mode and VU-mode External Debug Security Extension (Smuedbgsec)	21
U-mode External Debug Control	21
VU-mode External Debug Control	22
udcsr and udpc	22
3.2. Trace Security Extensions	23
3.2.1. Trace Security Control States	23
3.2.2. M-mode Trace Security Extension (Smmetricsec)	24
3.2.3. S/HS-mode Trace Security Extension (Smsetrcsec)	24
3.2.4. VS-mode Trace Security Extension (Smvsetrcsec)	24
3.2.5. U-mode and VU-mode Trace Security Extension (Smuetrcsec)	25
4. Smtdeleg and Sstcfg (ISA Extensions)	27
4.1. Sstcfg Scope	27
4.2. Supervisor Trigger Select (stselect)	27
4.3. Supervisor and Virtual Supervisor Trigger Access	28
4.3.1. Virtualization	28
4.4. Supervisor Trigger State Access Modifications	29
4.4.1. Type Delegation Constraints	29
4.4.2. DMODE Interaction	30
4.4.3. HS-mode Management of VS/VU Trigger Context	30
4.5. Supervisor and Virtual Supervisor Trigger State Access Control	31
4.6. Access to tcontrol	32

4.7. Debugger Access from S/HS-mode and VS-mode.....	32
4.8. Trigger Allocation.....	32
4.9. Dependencies.....	33
4.10. Summary of New State.....	33
5. Sucsind (ISA Extension)	35
5.1. User-level CSRs.....	35
5.2. Virtualization	36
5.3. Access Control by the State-Enable CSRs.....	36
5.4. Dependencies.....	36
5.5. Summary of New State	37
6. Sutcfg (ISA Extension).....	39
6.1. User Trigger Select (utselect).....	39
6.2. User Trigger Access	39
6.2.1. Virtualization	40
6.3. User Trigger State Access Modifications.....	40
6.3.1. Type Delegation Constraints	40
6.3.2. DMODE Interaction	41
6.4. User Trigger State Access Control.....	41
6.5. Access to tcontrol	42
6.6. Debugger Access from U-mode and VU-mode.....	43
6.7. Trigger Allocation	43
6.8. Dependencies.....	43
6.9. Summary of New State	44
7. Debug Module Security (non-ISA) Extension.....	45
7.1. External Debug Security Extensions Discovery	45
7.2. Halt	45
7.3. Reset.....	45
7.4. Keepalive.....	45
7.5. Abstract Commands.....	46
7.5.1. Relaxed Permission Check relaxedpriv	46
7.5.2. Address Translation AAMVIRTUAL	46
7.5.3. Quick Access	46
7.6. System Bus Access.....	46
7.7. Security Fault Error Reporting.....	46
7.8. Platform Debug Security Enable	47
7.9. Authorization Handoff.....	47
7.10. Update of Debug Module Registers.....	48
Appendix A: Valid Extension Combinations.....	49
A.1. External Debug Security Extension Combinations	49
A.2. Trace Security Extension Combinations.....	49
Appendix B: Software Debug/Trace Security Policy Management.....	51
B.1. M-mode Management	51
B.2. S/HS-mode Management	51
B.3. VS-mode Management.....	52
B.4. U-mode and VU-mode Management.....	52

Appendix C: Execution-Based Implementation with Sdsec.....	53
Bibliography.....	55

Preamble



This document is in the *Development state*

Expect potential changes. This draft specification is likely to evolve before it is accepted as a standard. Implementations based on this draft may not conform to the future standard.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2023-2024 by RISC-V International.

Contributors

This RISC-V specification has been contributed to directly or indirectly by (in alphabetical order): Allen Baum, Aote Jin (editor), Beeman Strong, Erik Kraft, Gokhan Kaplayan, Greg Favor, Iain Robertson, Joe Xie (editor), Paul Donahue, Ravi Sahita, Robert Chyla, Thomas Roecker, Tim Newsome, Ved Shanbhogue, Vicky Goode

Chapter 1. Introduction

Debugging and tracing are essential for developers to identify and rectify software and hardware issues, optimize performance, and ensure robust system functionality. The debugging and tracing extensions in the RISC-V ecosystem play a pivotal role in enabling these capabilities, allowing developers to monitor and control the execution of programs during the development, testing, and production phases. However, the current RISC-V Debug specification grants the external debugger the highest privilege in the system, regardless of the privilege level at which the target system is running, and tracing is also allowed in all privilege modes. This leads to privilege escalation and information leakage issues when multiple actors are present.

This specification defines [Debug Module Security Extension \(non-ISA extension\)](#) and [Secure Debug and Secure Trace ISA extensions](#) (collectively referred to as Sdsec) to address the above security issues in the current *The RISC-V Debug Specification* [1] and trace specifications [2] [3]. It also defines the distinct trigger-delegation ISA extensions [Smtdeleg](#), [Sstcfg](#), [Sucsrind](#), and [Sutcfg](#), which provide S/HS-mode, VS-mode, and U/VU-mode access to Sdtrig trigger state.

At a high level, the Debug Module Security Extension provides a platform-level control state, **psecdngen**, to enable or bypass the debug security constraints. The Secure Debug and Secure Trace ISA extensions provide per-hart debug and trace control states, including platform-controlled **mdngen** and **mtrcen** states for M-mode and lower-privilege control states in **mdtcfg**. Together, these control states determine whether external debug or trace is allowed at the hart's current privilege mode. **psecdngen** does not control trace. Implementation of either ISA family is optional even when the corresponding external debug or trace facility is present.

A summary of the changes introduced by *The RISC-V External Debug Security Specification* follows.

- **Per-Hart Debug Control:** Introduce per-hart states to control whether external debug is allowed for each privilege mode.
- **Per-Hart Trace Control:** Introduce per-hart states to control whether tracing is allowed for each privilege mode.
- **Platform debug security enable:** Add a platform state to enable the security constraints or bypass them for backward compatibility.
- **Debug Mode entry:** An external debugger can only halt the hart and enter Debug Mode when external debug is allowed in the current privilege mode.
- **Memory Access:** Memory access from a hart's point of view, using the Program Buffer or an Abstract Command, must be checked by the hart's memory protection mechanisms as if the hart is running at [debug access privilege level](#); memory access from the Debug Module using System Bus Access (SBA) must be checked by a system memory protection mechanism, such as IOPMP or RISC-V Worlds.
- **Register Access:** Register access using the Program Buffer or an Abstract Command works as if the hart is running at [debug access privilege level](#) instead of the M-mode privilege level. The debug CSRs (**dcsr** and **dpc**) are shadowed in lower privilege modes.
- **Trigger Delegation:** [Smtdeleg](#) and [Sstcfg](#) provide S/HS-mode and VS-mode access to Sdtrig triggers, and [Sucsrind](#) and [Sutcfg](#) provide U-mode and VU-mode access, when Debug Mode has less than M-mode privilege.

1.1. Control Hierarchy

Operationally, **psecdngen** selects between backward-compatible debug behavior and the debug security constraints defined by this specification. When those constraints are enabled, each hart evaluates its debug control states for its current privilege mode. Debug operations are permitted only when the corresponding

debug control state allows access at that mode; otherwise the request is rejected or remains pending. Trace output is independent of `psebdbgen` and is enabled only for modes allowed by the trace control states.

1.1.1. External Debug Security Control

The debug security policy for a hart is enforced through a hierarchical control model using the platform-controlled `mdbgen` state for M-mode and optional lower-privilege control states (`SEDBGEN`, `VSEDBGEN`, `UEDBGEN`, `VUEDBGEN`) in the `mdtcf` CSR. The `mdbgen` state can be implemented as an input signal to the hart and managed through platform-specific mechanisms such as Debug Module registers, a Root of Trust (RoT) handshake, or lifecycle fuses. The `psebdbgen` platform control state enables the debug security policy at the platform level; when `psebdbgen` is 0, the platform provides unrestricted debug access for all privilege modes.

The debug security control logic validates all debug requests and triggers whose `ACTION=1` against the hart's current privilege level and the configured debug security control states. The validation procedure involves two steps: (1) checking `psebdbgen` to determine whether the debug security policy is enabled and (2) checking the hierarchical control states to determine whether external debug is allowed at the hart's current privilege level. Debug requests that fail validation are either dropped or kept pending until the hart transitions to a debug-allowed privilege mode. [External Debug Security Extensions](#) defines the detailed behavior.

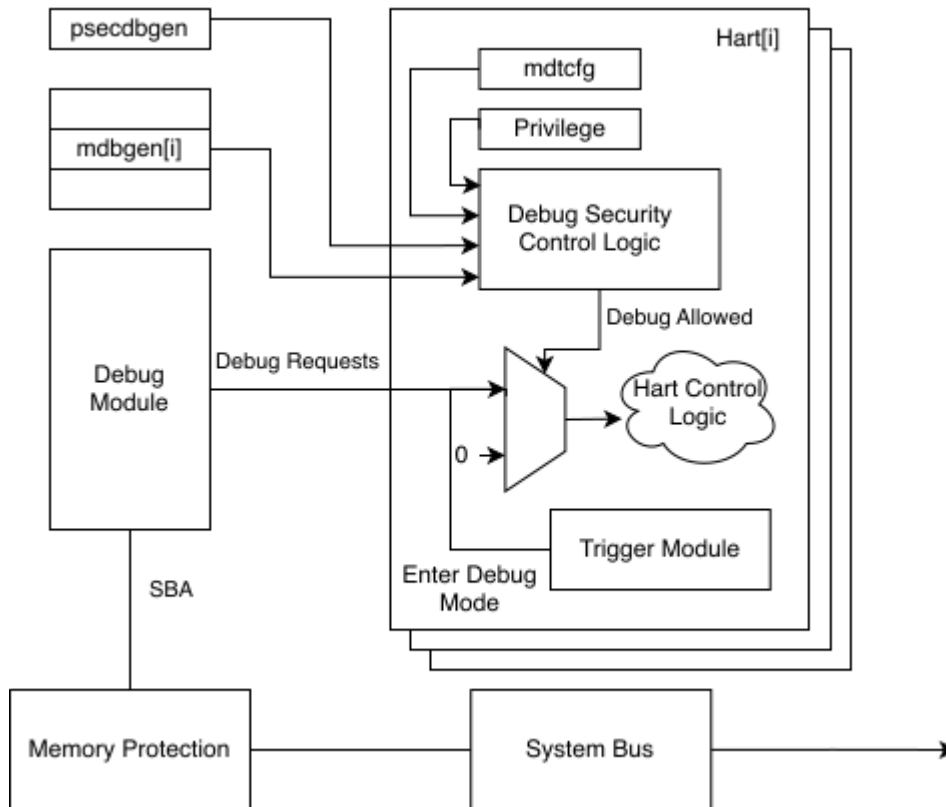


Figure 1. The debug security control for one hart

The hierarchical control applies as follows for a single hart:

- When `psebdbgen`=0, external debug is allowed for all privilege modes.
- When `psebdbgen`=1 and `mdbgen`=1, external debug is allowed for all privilege modes.
- When `psebdbgen`=1, `mdbgen`=0, and `SEDBGEN`=1 (if implemented), external debug is allowed for S/HS-mode and lower privilege modes.
- When `psebdbgen`=1, `mdbgen`=0, `SEDBGEN`=0, and `VSEDBGEN`=1 (if implemented), external debug is allowed for VS-mode and VU-mode.

- When **psebdbgen**=1, **mdbgen**=0, **SEDBGGEN**=0, and **UEDBGEN**=1 (if implemented), external debug is allowed for U-mode. **VSEDBGGEN** and **VUEDBGEN** do not affect U-mode debug permission.
- When **psebdbgen**=1, **mdbgen**=0, **SEDBGGEN**=0, **VSEDBGGEN**=0, and **VUEDBGEN**=1 (if implemented), external debug is allowed for VU-mode. **UEDBGEN** does not affect VU-mode debug permission.
- External debug is disallowed in a privilege mode when none of the control states applicable to that mode allow debug.

The debug security control logic must ensure that debug requests are validated and executed at the same privilege level and with the same debug security control states to prevent time-of-check to time-of-use (TOCTOU) vulnerabilities. Failing to do so may allow debug requests to bypass security controls.



Self-hosted debugging is also commonly used for application-level debugging, providing a pure software-based solution that does not require an external debugger. It is orthogonal to external debugging and is not affected by the debug security controls.

1.1.2. Trace Security Control

The trace security policy for a hart is enforced through a hierarchical control model using the platform-controlled **mtrcen** state for M-mode and optional lower-privilege control states (**SETRCEN**, **VSETRCEN**, **UETRCEN**, **VUETRCEN**) in the **mdtcfg** CSR. The **mtrcen** state can be implemented as an input signal to the hart and managed through platform-specific mechanisms such as a Root of Trust (RoT) handshake or lifecycle fuses. Trace has no privilege mode above M-mode, so this specification does not introduce a platform-level enable analogous to **psebdbgen**; **psebdbgen** does not control trace.

The security control logic determines whether trace is allowed based on the hart's current privilege level and the configured trace security control states. The **sec_inhibit** sideband signal to the trace encoder is asserted when trace is not allowed; equivalently, **sec_inhibit** is the inverse of the trace-allowed decision shown in the reference control logic. [Trace Security Extensions](#) defines the detailed behavior.

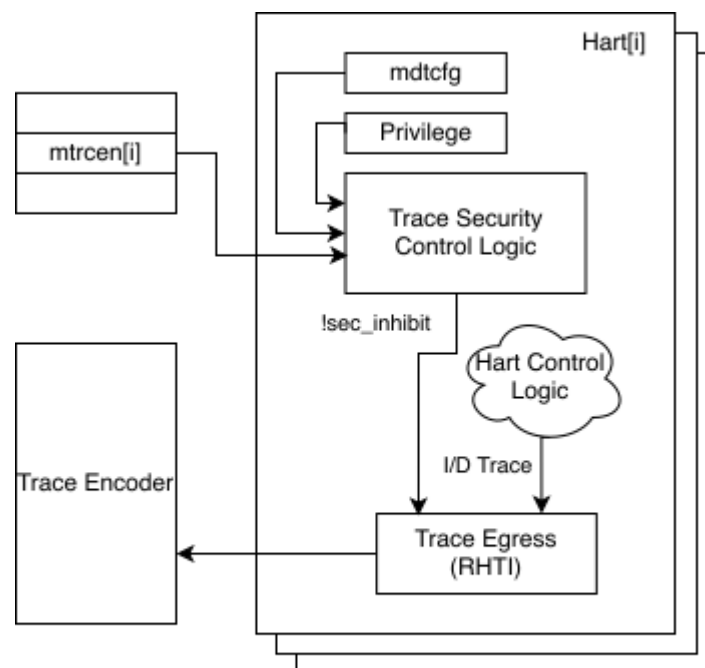


Figure 2. The trace security control for one hart

The hierarchical control applies as follows for a single hart:

- When **mtrcen**=1, trace is allowed for all privilege modes.
- When **mtrcen**=0 and **SETRCEN**=1 (if implemented), trace is allowed for S/HS-mode and lower privilege modes; trace is inhibited for M-mode.
- When **mtrcen**=0, **SETRCEN**=0, and **VSETRCEN**=1 (if implemented), trace is allowed for VS-mode and VU-mode; trace is inhibited for M-mode and S/HS-mode.
- When **mtrcen**=0, **SETRCEN**=0, and **UETRCEN**=1 (if implemented), trace is allowed for U-mode. **VSETRCEN** and **VUETRCEN** do not affect U-mode trace permission.
- When **mtrcen**=0, **SETRCEN**=0, **VSETRCEN**=0, and **VUETRCEN**=1 (if implemented), trace is allowed for VU-mode. **UETRCEN** does not affect VU-mode trace permission.
- When all trace control states are 0, trace is completely inhibited for all privilege modes.

1.2. Terminology

Abstract command	A high-level Debug Module operation used to interact with and control harts
Debug Access Privilege	The privilege level with which an Abstract Command or instruction in the Program Buffer accesses hardware resources
Debug Mode	An additional privilege mode to support off-chip debugging
Hart	A RISC-V hardware thread
IOPMP	Input-Output Physical Memory Protection unit
M-mode	The highest privileged mode in the RISC-V privilege model
PMA	Physical Memory Attributes
PMP	Physical Memory Protection unit
Program buffer	A mechanism that allows the Debug Module to execute arbitrary instructions on a hart
RISC-V Worlds	A system-level isolation mechanism that constrains physical-address access by harts and other bus initiators
Trace encoder	A piece of hardware that takes in instruction execution information from a RISC-V hart and transforms it into trace packets

Chapter 2. External Debug Security Threat Model

Modern SoC development involves several different actors who may not trust each other, resulting in the need to isolate actors' assets during the development and debugging phases. The current RISC-V Debug specification [1] grants external debuggers the highest privilege in the system, regardless of the privilege level at which the target system is running. This leads to privilege escalation issues when multiple actors are present.

For example, the owner of an SoC, who needs to debug their firmware, may be able to use the external debugger to bypass PMP (including PMP lock `pmpecfg.L=1`) and MMU protections, attacking Boot ROM or other SoC creator's assets.

Additionally, the RISC-V privilege architecture supports multiple software entities at different privilege levels that may not trust each other. These entities may be isolated from each other by mechanisms such as PMP/IOPMP or MMU, and may need different debug and trace policies. Since the external debugger is granted the highest privilege in the system, a malicious entity at a lower privilege level could compromise higher privilege software with the external debugger. Similarly, trace output from higher privilege execution could leak sensitive information to entities at lower privilege levels if not properly controlled.

Chapter 3. Secure Debug and Secure Trace (ISA Extensions)

This chapter specifies ISA extensions that add security controls to external debug (Sdext/Sdtrig) [1] and to trace [2] [3]. Sdsec is not an extension of its own; it is the umbrella name for two families of extensions: Secure Debug and Secure Trace.

The Secure Debug family provides hierarchical privilege-based control of external debug. When this family is implemented, the following extensions are defined:

- **Smmedbgsec (Base)**: M-mode external debug control
- **Smsedbgsec (Optional)**: Extends Smmedbgsec to add S/HS-mode external debug control
- **Smvsedbgsec (Optional)**: Adds VS-mode external debug control
- **Smuedbgsec (Optional)**: Adds independent U-mode and VU-mode external debug controls

The Secure Trace family provides hierarchical privilege-based control of trace output. When this family is implemented, the following extensions are defined:

- **Smmetricsec (Base)**: M-mode trace control
- **Smsetricsec (Optional)**: Extends Smmetricsec to add S/HS-mode trace control
- **Smvsetricsec (Optional)**: Adds VS-mode trace control
- **Smuetricsec (Optional)**: Adds independent U-mode and VU-mode trace controls

Each family is optional and independent of the other: an implementer may implement the Secure Debug family, the Secure Trace family, or both. Implementing external debug or trace does not by itself require the corresponding family. Implementations may include some or all optional extensions of an implemented family. When an optional extension for a lower privilege mode is implemented, all extensions for implemented higher privilege modes of the same family must also be implemented. These extensions establish hierarchical controls that can restrict external debug access or trace output to specific privilege levels.

Smtdeleg, Sstcfg, Sucsrind, and Sutcfg are distinct extensions specified in [Chapter 4](#), [Chapter 5](#), and [Chapter 6](#). They are not members of either family. Sstcfg requires Smtdeleg. Sutcfg requires Sucsrind and, when S-mode is implemented, also requires Smtdeleg and Sstcfg. When implemented, these extensions provide S/HS-mode, VS-mode, and U/VU-mode access to Sdtrig trigger state for a lower-privilege debugger. Neither Secure Debug nor Secure Trace requires these extensions.

3.1. External Debug Security Extensions

External debug operations can modify system state and expose sensitive information. The following extensions provide hierarchical privilege-based controls to restrict debug access, preventing unauthorized debugging of higher-privilege software.

3.1.1. External Debug Security Control States and CSRs

The following table summarizes the external debug control states and associated CSRs introduced by the Debug Module Security Extension and the Sdsec external debug security extensions. In this specification, *control state* covers platform-level control state, such as **psecdbgen**, per-hart platform-controlled states, such as **mdbgen**, and CSR fields, such as **mdtcfg.SEDBGEN**.

Table 1. External Debug Control States and CSRs

Extension	External Debug Control State	New CSRs	Scope
Debug Module Security Extension	psecdbggen	None	Platform/all modes
Smmedbgsec	mdbggen	mdtcfg	M-mode
Smsedbgsec	mdtcfg.SEDBGEN	sdcsr, sdpc	S/HS-mode
Smvsedbgsec	mdtcfg.VSEDBGEN	None	VS-mode
Smuedbgsec	mdtcfg.UEDBGEN, mdtcfg.VUEDBGEN	udcsr, udpc	U-mode and VU-mode, independently

When external debug is allowed for a privilege mode, it is allowed for all lower privilege modes. U-mode and VU-mode are not ordered with respect to each other, so **UEDBGEN** and **VUEDBGEN** control them independently. If an optional debug extension is not implemented, the corresponding control state behaves as read-only 0; for lower-privilege modes, the **mdtcfg** field representing that control state is also read-only 0.

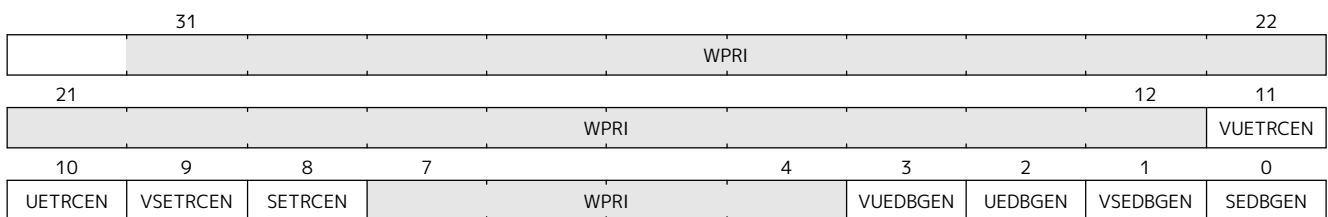
The **mdtcfg** (Machine Debug and Trace Configuration) is a machine-mode read/write CSR (address TBD). It is a single CSR shared by the Secure Debug and Secure Trace families. Debug-related fields (**SEDBGEN**, **VSEDBGEN**, **UEDBGEN**, **VUEDBGEN**) and trace-related fields (**SETRCEN**, **VSETRCEN**, **UETRCEN**, **VUETRCEN**) are independent: an implementation may implement Secure Debug, Secure Trace, or both without coupling the two functions through this CSR.

mdtcfg is implemented if Smmedbgsec, Smmetrcsec, or any optional extension in either family is implemented. Fields associated with an unimplemented family or optional extension are WARL and read-only 0.

The M-mode external debug and trace enables are the platform-controlled **mdbggen** and **mtrcen** states. They are not fields of **mdtcfg** and are not writable by M-mode software. Writing **mdtcfg** therefore cannot enable or disable M-mode external debug or M-mode trace; M-mode software uses **mdtcfg** only to configure lower-privilege debug and trace enables.

Table 2. Machine Debug and Trace Configuration CSR

Number	Privilege	Width	Name	Description
address TBD	MRW	XLEN	mdtcfg	Machine debug and trace configuration



Register 1: **mdtcfg** register

The following fields in **mdtcfg** provide external debug control states for privilege modes other than M-mode. The trace control states are described in [Section 3.2](#).

Table 3. Details of external debug control states in **mdtcfg**

Field	Description	Access	Reset
SEDBGEN	S/HS-mode external debug enable. When set to 1, external debug is allowed for S/HS-mode, U-mode, VS-mode, and VU-mode when mdbggen is 0. Introduced by Smsedbgsec.	WARL	0

Field	Description	Access	Reset
VSEDBGEN	VS-mode external debug enable. When set to 1, external debug is allowed for VS-mode and VU-mode when mdbgen =0 and SEDBGEN =0. This field does not control U-mode. Introduced by Smvsedbgsec.	WARL	0
UEDBGEN	U-mode external debug enable. When set to 1, U-mode external debug is allowed when mdbgen =0 and SEDBGEN =0. VSEDBGEN and VUEDBGEN do not affect U-mode permission. Introduced by Smuedbgsec.	WARL	0
VUEDBGEN	VU-mode external debug enable. When set to 1, VU-mode external debug is allowed when mdbgen =0, SEDBGEN =0, and VSEDBGEN =0. UEDBGEN does not affect VU-mode permission. Introduced by Smuedbgsec.	WARL	0



The bit positions shown above are tentative and have not been officially allocated. The final bit assignments will be determined during the ratification process.

3.1.2. Debug Operations

Chapter 3 of *The RISC-V Debug Specification* [1] outlines all mandatory and optional external debug operations. The operations listed below are affected by the Secure Debug extensions; other operations remain unaffected. In the context of this chapter, **debug operations** refer to those listed below.

Debug operations affected by the Secure Debug extensions:

- Halting the hart to enter Debug Mode
- Executing the Program Buffer
- Serving abstract commands (Access Register, Access Memory)

3.1.3. Debug Access Privilege

The **debug access privilege** is the privilege level used for state accesses performed by the Debug Module or through the Program Buffer, including accesses performed by Abstract Commands and Program Buffer instructions. Register and memory accesses use the privilege level derived in Table 4. Attempts to access state that requires a privilege level above the **debug access privilege** fail and set **abstractcs.CMDERR** to 3.

Table 4. External Debug Configuration and Privilege

mdbgen	SEDBGEN	VSEDBGEN	UEDBGEN	VUEDBGEN	Debugged Context	Debug Access Privilege
1	Don't care	Don't care	Don't care	Don't care	Any supported mode	M-mode
0	1	Don't care	Don't care	Don't care	S/HS-mode, VS-mode, U-mode, or VU-mode	S/HS-mode
0	0	1	Don't care	Don't care	VS-mode or VU-mode	VS-mode
0	0	Don't care	1	Don't care	U-mode	U-mode
0	0	0	Don't care	1	VU-mode	VU-mode

3.1.4. Allowed Resume Privilege Modes

The privilege modes that can be configured in **dcscr.PRIV** and **dcscr.V** are determined by the table below. If an unsupported or disallowed value is written, the fields retain legal values.

Table 5. Allowed Resume Privilege Modes

Resume Mode	Required Debug Control State
M-mode	mdbggen =1
S/HS-mode	mdbggen =1, or mdbggen =0 and SEDBGGEN =1
VS-mode	mdbggen =1, or mdbggen =0 and either SEDBGGEN =1 or VSEDBGGEN =1
U-mode	mdbggen =1, or mdbggen =0 and either SEDBGGEN =1 or UEDBGEN =1
VU-mode	mdbggen =1, or mdbggen =0 and at least one of SEDBGGEN , VSEDBGGEN , or VUEDBGEN is 1

3.1.5. M-mode External Debug Security Extension (Smmedbgsec)

The Smmedbgsec extension is the base extension of the Secure Debug family.



The **mdbggen** state is a per-hart platform-controlled state that controls whether M-mode external debug capabilities are available. It may be delivered to the hart through various methods, such as a new input port, a handshake with the system Root of Trust (RoT), or other methods. The **mdbggen** state for the Root of Trust (RoT) itself should be managed by SoC hardware, likely depending on lifecycle fusing. The implementation can choose to group several harts together and use one signal to drive their **mdbggen** state or assign each hart its own dedicated state. For example, a homogeneous computing system can use a signal to drive all **mdbggen** states to enforce a unified debug policy across all harts.

When **mdbggen** is set to 1, debug is allowed for M-mode and the following rules apply:

- The [debug access privilege](#) for the hart is M-mode. Abstract Commands, including Quick Access, and instructions in the Program Buffer operate with M-mode privilege.
- The [debug operations](#) are allowed when the hart executes in any privilege mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting **abstractcs.CMDERR** to 3.
- The [allowed resume privilege modes](#) include M-mode and all lower privilege modes.

When **mdbggen** is set to 0 and the hart is running in M-mode, external debug is disallowed for M-mode:

- The hart will not enter Debug Mode:
 - Halt requests will remain pending until external debug is allowed.
 - Triggers with ACTION=1 (enter Debug Mode) will not match or fire.
 - EBREAK cannot enter Debug Mode and always raises a breakpoint exception.
- The external trigger outputs (with ACTION=8/9) will not match or fire.
- Accesses to M-mode accessible CSRs (e.g., **dcscr**, **dpc**, and **dscratch0/1**) will fail and **abstractcs.CMDERR** is set to 3 (exception).
- The configuration in **dcscr** will be ignored as if it were 0.
- DMODE is read/write for accesses from M-mode when Sdtrig is implemented.

When **mdbggen**=0 and the hart is not in Debug Mode, M-mode may read the native trigger CSRs.

If the selected trigger has **DMODE**=0, M-mode writes to **tdata1**, **tdata2**, and **tdata3** follow the base Sdtrig rules.

If the selected trigger has **DMODE**=1:

- Writes to **tdata2** and **tdata3** are ignored.
- For a write to **tdata1**, changes to **DMODE** and **ACTION** are permitted.
- A write of the disabled-trigger encoding (**TYPE**=15 and **DMODE**=0) is also permitted. The write shall be committed atomically as a disabled trigger, regardless of the previous **DMODE** value.
- Other fields in **tdata1** are ignored unless the write is the disabled-trigger write described above.
- If the resulting state would have **DMODE**=0 and **ACTION**=1, hardware shall set **ACTION** to 0.

When **mdbggen**=1, the native M-mode exception above does not apply: the native **DMODE** field is writable only from Debug Mode. This rule does not restrict the trusted S/HS-mode manager accesses through delegated lower-level views.



*This **DMODE** write access allows M-mode software to save and restore trigger context at lower-privilege software boundaries. After setting **DMODE**=0, software should disable the trigger or clear its mode-enable bits before a lower-privilege context runs. To restore a saved **DMODE**=1 configuration, write the saved **tdata*** while **DMODE**=0, including **DMODE**=1 and **ACTION**=1 in a single **tdata1** write if that was the saved configuration. Without this access, an external debugger could leave **ACTION**=1 triggers that indirectly observe states (such as PC and data addresses) from the debug-disallowed software, potentially leading to information leakage or Denial of Service (DoS). When **mdbggen** is 1, **DMODE** remains accessible only from Debug Mode.*

S/HS-mode manager access to **DMODE**=1 trigger state for lower-privilege contexts is specified in [Section 4.4.3](#) and [Section 6.3.2](#). That access uses the applicable delegated view and does not provide S/HS-mode with native access to trigger state. The lower-privilege external debug enable fields control external debugger access and do not remove this trusted S/HS-mode manager access.

If the hart is running in a debug-allowed lower privilege mode (e.g., S/HS-mode) when **mdbggen** is 0:

- Single-stepping cannot stop in M-mode.
- Interrupts to M-mode cannot be disabled by setting **dcsr.STEPIE**=0.



*When **mdbggen**=0 and **dcsr.STEP**=1, a single-stepped instruction in a debug-allowed privilege mode may transfer control to the M-mode trap handler. The hart will execute the handler in M-mode and re-enter Debug Mode immediately after an MRET instruction returns to the debug-allowed privilege mode (i.e., MRET with **mstatus.MPP**<3). The hart does not re-enter Debug Mode if the MRET instruction returns to a debug-disallowed privilege mode (i.e., MRET with **mstatus.MPP**=3, **mdbggen**=0).*



*This specification assumes that the debugger ensures **mdbggen** shall never be set to 0 while the hart is in Debug Mode. Setting **mdbggen** to 0 while in Debug Mode could lead to undefined behavior; the hart may lose its debug privileges unexpectedly, potentially causing the debug session to fail or become insecure.*

3.1.6. S/HS-mode External Debug Security Extension (Smsedbgsec)

The Smsedbgsec extension is optional and requires Smmedbgsec. When [Smtdeleg](#) and [Sstcfg](#) are implemented, they provide the Sdtrig trigger access path for an S/HS-mode debugger. Smsedbgsec introduces the **SEDBGGEN** field in CSR **mdtcfg** and controls S/HS-mode debuggability when M-mode external

debug is disallowed.

The **SEDBGEN** field only takes effect when **mdbgen** is 0.

When **SEDBGEN** is set to 1, S/HS-mode external debug is allowed:

- The **debug access privilege** for the hart is S/HS-mode. Abstract Commands, including Quick Access, and instructions in the Program Buffer operate with S/HS-mode privilege.
- The **debug operations** are allowed when the hart executes in S/HS-mode, U-mode, VS-mode, or VU-mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting **abstractcs.CMDERR** to 3.
- The **sdcsr** and **sdpc** registers (Section 3.1.6.1) are introduced in Smsgdbgsec to provide an S/HS-mode debugger with access to **dcsr** and **dpc**.
- When Smtdeleg and Sstcfg are implemented, an S/HS-mode debugger can access Sdtrig trigger state through the **Smtdeleg/Sstcfg** interface (**stselect** and **siselect/sireg***).
- The **effective debug access privilege** of loads and stores in Debug Mode may be modified as described in Section 3.1.6.2.
- The **allowed resume privilege modes** include S/HS-mode and all lower privilege modes.

When **SEDBGEN** is set to 0 and the hart is running in S/HS-mode, external debug is disallowed for S/HS-mode:

- The hart will not enter Debug Mode while running in S/HS-mode:
 - Halt requests will remain pending until external debug is allowed.
 - Triggers with ACTION=1 (enter Debug Mode) will not match or fire.
 - EBREAK cannot enter Debug Mode and always raises a breakpoint exception.
- The external trigger outputs (with ACTION=8/9) will not match or fire while in S/HS-mode.
- Accesses to S/HS-mode accessible CSRs (e.g., **sdcsr** and **sdpc**) will fail and **abstractcs.CMDERR** is set to 3 (exception).
- The configuration visible in **sdcsr** will be ignored as if it were 0.

When **SEDBGEN** is set to 0 and the hart is running in a debug-allowed lower privilege mode (e.g., U-mode), the following restrictions apply:

- Single-stepping cannot stop in S/HS-mode.
- Interrupts delegated to S/HS-mode cannot be disabled by setting **dcsr.STEPIE**=0.



When **SEDBGEN**=0 and **sdcsr.STEP**=1, a single-stepped instruction in a debug-allowed privilege mode may transfer control to the S/HS-mode trap handler. The hart will execute the handler in S/HS-mode and re-enter Debug Mode immediately after an SRET instruction returns to the debug-allowed privilege mode (i.e., SRET with **sstatus.SPP**=0). The hart does not re-enter Debug Mode if the SRET instruction returns to a debug-disallowed privilege mode.

sdcsr and sdpc

When **SEDBGEN** is 1, the **sdcsr** and **sdpc** registers provide S/HS-mode read/write access to the **dcsr** and **dpc** registers, respectively. However, **sdcsr** does not expose access to the **MPRVEN** field; instead, it repurposes the **MPRVEN** bit position as a **DMPRV** field that modifies the **effective debug access privilege** in S/HS-mode. Both

registers are accessible only in Debug Mode.

Table 6. CSR addresses for S/HS-mode shadows of Debug Mode CSRs

Number	Name	Description
address TBD	sdcsr	S/HS-mode debug control and status register.
address TBD	sdpc	S/HS-mode debug program counter.

The **sdcsr** register exposes a subset of **dcsr**, formatted as shown in [Register 2](#), while the **sdpc** register provides full access to **dpc**.



Unlike **dcsr** and **dpc**, the **dscratch0/1** registers do not have an S/HS-mode access mechanism, and external debuggers with S/HS-mode privilege cannot use them. In The RISC-V Debug Specification [1], **dscratch*** are optional scratch registers that implementations may use internally; **hartinfo.NSCRATCH** indicates how many are available, and a debugger must not write to these registers unless **hartinfo** explicitly mentions them. Compliant debug flows already work without **dscratch** (for example, Access Register, GPRs, and data registers). Sdsec shadows only the required control state at each privilege level (**dcsr/dpc** → **sdcsr/sdpc** and, when applicable, **udcsr/udpc**); no **sdscratch** registers are defined. In an execution-based implementation, the park loop runs at M-mode and may use existing **dscratch** registers, while Abstract Commands and the Program Buffer run at the [debug access privilege](#).

31	28	27	26	24	23	22
0	0	0	0	0	0	0
21	19	18	17	16	15	14
0	0	PELP	EBREAKVS	EBREAKVU	0	EBREAKS
10	9	8	6	5	4	3
0	0	CAUSE	V	DMPRV	0	STEP
1	0	0	PRV	0	0	0

Register 2: S/HS-mode debug control and status register (**sdcsr**)



The **NMIP**, **MPRVEN**, **STOPTIME**, **STOPCOUNT**, **EBREAKM**, and **CETRIG** fields in **dcsr** are configurable only by M-mode; they are masked in **sdcsr**, while **PRV[1]** is hardwired to 0 in **sdcsr**. The field for **MPRVEN** is reclaimed by **DMPRV** in the **sdcsr** layout to avoid wasting fields.

Table 7. Details of the **dmprv** field in **sdcsr**

Field	Description	Access	Reset
DMPRV	0 (normal): The privilege level in Debug Mode is not modified. 1: In Debug Mode, the privilege level for load and store operations is modified to the effective debug access privilege as described in Section 3.1.6.2 and Section 3.1.7.2 .	WARL	0

Debug Access Privilege for Memory in Smedbgsec

The **sdcsr.DMPRV** field takes effect when **mdbggen** is 0 and is read-only 0 when **mdbggen** is 1. When **SEDBGGEN** is 1, **sdcsr.DMPRV** can modify the **effective debug access privilege** used by an S/HS-mode debugger's loads and stores in Debug Mode. When **sdcsr.DMPRV**=0, the **effective debug access privilege** of loads and stores in Debug Mode follows [Table 4](#); when **sdcsr.DMPRV**=1, the **effective debug access privilege** of loads and stores in Debug Mode is represented by **sstatus.SPP** and **hstatus.SPV** (if the hypervisor extension is supported).

The **sdcsr.DMPRV** does not affect the virtual-machine load/store instructions, **HLV**, **HLVX**, and **HSV**.

3.1.7. VS-mode External Debug Security Extension (Smvsdbgsec)

The Smvsdbgsec extension is optional and requires Smsdbgsec. When Smtdeleg and Sstcfg are implemented, a VS-mode debugger can use the same delegated trigger interface with the Sscsrind substitution to **vsiselect/vsireg***. Smvsdbgsec introduces the **VSEDBGEN** field in CSR **mdtcfg** and controls VS-mode debuggability when S/HS-mode external debug is disallowed.

The **VSEDBGEN** field only takes effect when both **mdbgen** is 0 and **SEDBGEN** is 0.

When **VSEDBGEN** is set to 1, VS-mode external debug is allowed:

- The [debug access privilege](#) for the hart is VS-mode. Abstract Commands, including Quick Access, and instructions in the Program Buffer operate with VS-mode privilege.
- The [debug operations](#) are allowed when the hart executes in VS-mode or VU-mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting **abstractcs.CMDERR** to 3.
- The **sdcsr** and **sdpc** registers (from Smsdbgsec) provide VS-mode read/write access to **dcsr** and **dpc**, respectively, as specified in [Section 3.1.7.1](#).
- The **effective debug access privilege** of loads and stores in Debug Mode may be modified as described in [Section 3.1.7.2](#).
- The [allowed resume privilege modes](#) include VS-mode and VU-mode.

When **VSEDBGEN** is set to 0 and the hart is running in VS-mode, external debug is disallowed for VS-mode:

- The hart will not enter Debug Mode while running in VS-mode:
 - Halt requests will remain pending until external debug is allowed.
 - Triggers with ACTION=1 (enter Debug Mode) will not match or fire.
 - EBREAK cannot enter Debug Mode and always raises a breakpoint exception.
- The external trigger outputs (with ACTION=8/9) will not match or fire while in VS-mode.
- Accesses to VS-mode accessible CSRs (e.g., **sdcsr** and **sdpc**) will fail and **abstractcs.CMDERR** is set to 3 (exception).
- The configuration visible to VS-mode through **sdcsr** will be ignored as if it were 0.

When **VSEDBGEN** is set to 0 and the hart is running in a debug-allowed VU-mode, the following restrictions apply:

- Single-stepping cannot stop in VS-mode.
- Interrupts delegated to VS-mode cannot be disabled by setting **sdcsr.STEPIE**=0.



*When **VSEDBGEN**=0 and **sdcsr.STEP**=1, a single-stepped instruction in a debug-allowed VU-mode may transfer control to the VS-mode trap handler. The hart will execute the handler in VS-mode and re-enter Debug Mode immediately after an SRET instruction returns to the debug-allowed VU-mode. The hart does not re-enter Debug Mode if the SRET instruction returns to a debug-disallowed VS-mode.*

sdcsr and **sdpc** in VS-mode

When **VSEDBGEN** is 1, the **sdcsr** and **sdpc** ([Section 3.1.6.1](#)) are accessible with VS-mode privilege, providing access to the **dcsr**. The **sdcsr.EBREAKS** and **sdcsr.EBREAKU** fields are redirected to **dcsr.EBREAKVS** and

`dcscr.EBREAKVU`, while writes to `sdcsr.EBREAKVS`, `sdcsr.EBREAKVU`, and `sdcsr.V` are discarded (writes are ignored and reads return 0). Similar to `sdcsr` access when `SEDBGEN` is 1, `sdcsr.DMPRV` modifies the effective debug access privilege in VS-mode.



Redirected access to `dcscr.EBREAKVS` and `dcscr.EBREAKVU` unifies the configuration for both S/HS-mode and VS-mode. However, the virtualization mode cannot be changed through `sdcsr.V` by a VS-mode debugger.

Debug Access Privilege for Memory in Smvsedbgsec

The `sdcsr.DMPRV` modifies the effective debug access privilege of loads and stores for a VS-mode debugger when `SEDBGEN` is 0 and `VSEDBGEN` is 1.

When `sdcsr.DMPRV`=0, the effective debug access privilege of loads and stores in Debug Mode follows Table 4; when `sdcsr.DMPRV`=1, the effective debug access privilege of loads and stores in Debug Mode is represented by `vsstatus.SPP` with the virtualization mode being honored as 1.

3.1.8. U-mode and VU-mode External Debug Security Extension (Smuedbgsec)

The Smuedbgsec extension is optional. It requires Smmedbgsec and, on harts that implement S/HS-mode, Smsedbgsec. When VU-mode control is implemented, Smuedbgsec also requires Smvsedbgsec. Smvsedbgsec is not required when only U-mode control is implemented and `VUEDBGEN` is read-only 0. Smuedbgsec introduces independent `UEDBGEN` and `VUEDBGEN` fields in CSR `mdtcfg`. If a mode or its corresponding control is not implemented, the corresponding field is read-only 0. When `Sutcfg` is implemented, it provides the Sdtrig trigger access path for a U-mode or VU-mode debugger.

U-mode External Debug Control

The `UEDBGEN` field takes effect when `mdbggen`=0 and `SEDBGEN`=0. Because U-mode and VS/VU-mode are not ordered with respect to each other, `VSEDBGEN` and `VUEDBGEN` do not affect U-mode external debug permission.

When `UEDBGEN`=1, `mdbggen`=0, and `SEDBGEN`=0, U-mode external debug is allowed:

- The [debug access privilege](#) is U-mode. Abstract Commands and instructions in the Program Buffer operate with U-mode privilege.
- The [debug operations](#) are allowed when the hart executes in U-mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting `abstractcs.CMDERR` to 3.
- The `udcsr` and `udpc` registers described in [Section 3.1.8.3](#) provide access to the permitted `dcscr` and `dpc` state.
- When `Sutcfg` is implemented, a U-mode debugger can access Sdtrig trigger state through the [Sutcfg](#) interface (`utselect` and `uiselect/uireg*`).
- U-mode is the only [allowed resume privilege mode](#).

When `UEDBGEN` is set to 0 and the hart is running in U-mode, external debug is disallowed for U-mode:

- The hart will not enter Debug Mode while running in U-mode:
 - Halt requests remain pending until external debug is allowed.
 - Triggers with ACTION=1 do not match or fire.

- EBREAK raises a breakpoint exception instead of entering Debug Mode.
- External trigger outputs with ACTION=8 or ACTION=9 do not match or fire.
- Accesses to user-visible debug CSRs fail and set **abstractcs.CMDERR** to 3.
- The configuration visible through **udcsr** is ignored as if it were 0.

VU-mode External Debug Control

The **VUEDBGEN** field takes effect when **mdbgen**=0, **SEDBGEN**=0, and **VSEDBGEN**=0. **UEDBGEN** does not affect VU-mode external debug permission.

When **VUEDBGEN**=1, **mdbgen**=0, **SEDBGEN**=0, and **VSEDBGEN**=0, VU-mode external debug is allowed:

- The [debug access privilege](#) is VU-mode. Abstract Commands and instructions in the Program Buffer operate with VU-mode privilege.
- The [debug operations](#) are allowed when the hart executes in VU-mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting **abstractcs.CMDERR** to 3.
- The **udcsr** and **udpc** registers described in [Section 3.1.8.3](#) provide access to the permitted **dcsr** and **dpc** state.
- When **Sutcfg** is implemented, a VU-mode debugger can access **Sdtrig** trigger state through the [Sutcfg](#) interface (**utselect** and **uiselect/uireg***).
- VU-mode is the only [allowed resume privilege mode](#).

When **VUEDBGEN** is set to 0 and the hart is running in VU-mode, external debug is disallowed for VU-mode:

- The hart will not enter Debug Mode while running in VU-mode:
 - Halt requests remain pending until external debug is allowed.
 - Triggers with ACTION=1 do not match or fire.
 - EBREAK raises a breakpoint exception instead of entering Debug Mode.
- External trigger outputs with ACTION=8 or ACTION=9 do not match or fire.
- Accesses to user-visible debug CSRs fail and set **abstractcs.CMDERR** to 3.
- The configuration visible through **udcsr** is ignored as if it were 0.

udcsr and udpc

The **udcsr** and **udpc** registers provide U-mode or VU-mode read/write access to the **dcsr** and **dpc** state when the corresponding **UEDBGEN** or **VUEDBGEN** control allows debug. The **udcsr** exposes the subset of **dcsr** shown in [Register 3](#). Access to **udcsr.EBREAKU** is redirected to **dcsr.EBREAKVU** when the virtualization mode is 1. The **udpc** register provides full access to **dpc**.

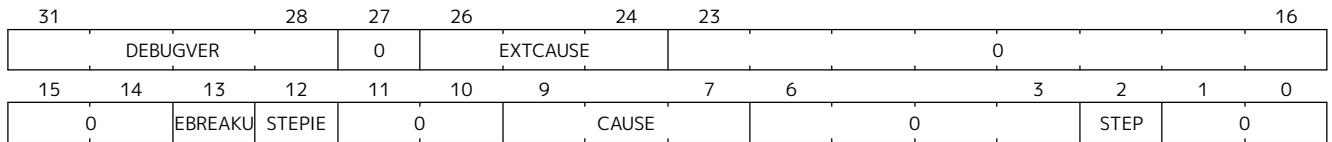


*The **udcsr** does not expose **dcsr.V**. A U-mode or VU-mode debugger therefore cannot switch between U-mode and VU-mode.*

Table 8. CSR addresses for U/VU-mode shadows of Debug Mode CSRs

Number	Name	Description
address TBD	udcsr	U-mode or VU-mode debug control and status register.
address TBD	udpc	U-mode or VU-mode debug program counter.

The **udcsr** register exposes a subset of **dcsr**, formatted as shown in [Register 3](#), while the **udpc** register provides full access to **dpc**.



Register 3: U-mode or VU-mode debug control and status register (udcsr)

3.2. Trace Security Extensions

When trace is implemented, trace output can expose sensitive information across privilege boundaries. The extensions below provide security controls to restrict trace output based on privilege levels.

3.2.1. Trace Security Control States

The following table defines the trace control states introduced by the trace security extensions. In this specification, *control state* covers both platform-controlled states, such as **mtrcen**, and CSR fields, such as **mdtcfg.SETRCEN**.

Table 9. Trace Control States

Extension	Trace Control State	Target Modes
Smmetrcsec	mtrcen	M-mode
Smsetrcsec	mdtcfg.SETRCEN	S/HS-mode
Smvsetrcsec	mdtcfg.VSETRCEN	VS-mode
Smuetrcsec	mdtcfg.UETRCEN, mdtcfg.VUETRCEN	U-mode and VU-mode, independently

Allowing trace for a privilege mode also allows it for all lower privilege modes. U-mode and VU-mode are not ordered with respect to each other, so **UETRCEN** and **VUETRCEN** control them independently. Each **mdtcfg** control field introduced by an unimplemented optional trace extension is read-only 0.



*The availability of trace output is controlled through signals defined in the RISC-V hart-trace interface (RHTI) [4]. An implementation can convert the logical **sec_inhibit** signal to the canonical trace interface signals.*

The following **mdtcfg** fields provide trace control states for privilege modes other than M-mode. The **mdtcfg** CSR must be implemented when **Smmetrcsec** is implemented, even if **Smmedbgsec** is not implemented.

Table 10. Details of trace security control states in mdtcfg

Field	Description	Access	Reset
SETRCEN	S/HS-mode external trace enable. When set to 1, trace is allowed for S/HS-mode, U-mode, VS-mode, and VU-mode when mtrcen is 0. Introduced by Smssetrcsec .	WARL	0
VSETRCEN	VS-mode external trace enable. When set to 1, trace is allowed for VS-mode and VU-mode when mtrcen =0 and SETRCEN =0. This field does not control U-mode. Introduced by Smvsetrcsec .	WARL	0

Field	Description	Access	Reset
UETRCEN	U-mode external trace enable. When set to 1, U-mode trace is allowed when mtrcen =0 and SETRCEN =0. VSETRCEN and VUETRCEN do not affect U-mode permission. Introduced by Smuetrcsec.	WARL	0
VUETRCEN	VU-mode external trace enable. When set to 1, VU-mode trace is allowed when mtrcen =0, SETRCEN =0, and VSETRCEN =0. UETRCEN does not affect VU-mode permission. Introduced by Smuetrcsec.	WARL	0

3.2.2. M-mode Trace Security Extension (Smmetricsec)

The Smmetricsec extension is the base extension of the Secure Trace family.



*The **mtrcen** state is a per-hart platform-controlled state that controls whether M-mode trace output is enabled. It may be delivered to the hart through various methods, such as a new input port, a handshake with the system Root of Trust (RoT), or other methods. The **mtrcen** state for the Root-of-Trust (RoT) itself should be managed by SoC hardware, likely dependent on lifecycle fusing. The implementation can choose to group several harts together and use one signal to drive their **mtrcen** state or assign each hart its own dedicated state. For example, a homogeneous computing system can use a signal to drive all **mtrcen** states to enforce a unified trace policy across all harts. Unlike external debug, trace has no privilege mode above M-mode, so this specification does not introduce a platform-level enable analogous to **psecdngen**. The **psecdngen** state does not control trace.*

When **mtrcen** is set to 1, trace is allowed for all privilege modes. When **mtrcen** is set to 0, trace is inhibited for M-mode by asserting the logical **sec_inhibit** signal to the trace encoder when the hart is executing in M-mode.

3.2.3. S/HS-mode Trace Security Extension (Smsetrcsec)

The Smsetrcsec extension is optional and requires Smmetricsec. It introduces the **SETRCEN** field in CSR **mdtcf** and controls S/HS-mode trace output when M-mode trace is disallowed.

The **SETRCEN** field only takes effect when **mtrcen** is 0.

When **SETRCEN** is set to 1, trace is allowed for all privilege modes except M-mode. When **SETRCEN** is set to 0, trace is inhibited for S/HS-mode by asserting the logical **sec_inhibit** signal to the trace encoder when the hart is executing in S/HS-mode.

3.2.4. VS-mode Trace Security Extension (Smvsetrcsec)

The Smvsetrcsec extension is optional and requires Smsetrcsec. It introduces the **VSETRCEN** field in CSR **mdtcf** and controls VS-mode trace output when S/HS-mode trace is disallowed.

The **VSETRCEN** field only takes effect when both **mtrcen** is 0 and **SETRCEN** is 0.

When **VSETRCEN** is set to 1, trace is allowed for VS-mode and VU-mode. When **VSETRCEN** is set to 0, trace is inhibited for VS-mode by asserting the logical **sec_inhibit** signal to the trace encoder when the hart is executing in VS-mode.

3.2.5. U-mode and VU-mode Trace Security Extension (Smuetrcsec)

The Smuetrcsec extension is optional. It requires Smmetricsec and, on harts that implement S/HS-mode, Smsetrcsec. When VU-mode trace control is implemented, Smuetrcsec also requires Smvsetrcsec. Smvsetrcsec is not required when only U-mode trace control is implemented and **VUETRCEN** is read-only 0. Smuetrcsec introduces independent **UETRCEN** and **VUETRCEN** fields in CSR **mdtcfg**. If a mode or its corresponding trace control is not implemented, the corresponding field is read-only 0.

The **UETRCEN** field takes effect when **mtrcen**=0 and **SETRCEN**=0. **VSETRCEN** and **VUETRCEN** do not affect U-mode trace permission. When **UETRCEN** is 1, trace is allowed in U-mode; when it is 0, trace is inhibited in U-mode by asserting the logical **sec_inhibit** signal.

The **VUETRCEN** field takes effect when **mtrcen**=0, **SETRCEN**=0, and **VSETRCEN**=0. **UETRCEN** does not affect VU-mode trace permission. When **VUETRCEN** is 1, trace is allowed in VU-mode; when it is 0, trace is inhibited in VU-mode by asserting the logical **sec_inhibit** signal.

Chapter 4. Smtdeleg and Sstcfg (ISA Extensions)

This chapter specifies two distinct ISA extensions that allow Sdtrig [1] triggers to be accessed from S/HS-mode and VS-mode. They are not part of the Secure Debug or Secure Trace families.

Sdtrig CSRs are accessible only with M-mode privilege. A self-hosted debugger with less than M-mode privilege therefore requires an SBI to reach them, which adds traps on trigger accesses. An external debugger has no SBI it can invoke from Debug Mode. If a Secure Debug extension is implemented and Debug Mode has less than M-mode privilege, the debugger cannot access triggers unless a delegated access path is provided.

The Smtdeleg extension provides that path for S/HS-mode and VS-mode, using the indirect CSR interface (Sschrind) [5]. It also controls S/HS-mode and VS-mode access to triggers using the State Enable extensions (Smstateen/Ssstateen) [6]. Sstcfg is the supervisor-level companion to Smtdeleg. Sstcfg is a distinct extension, and its implementation requires Smtdeleg.

U-mode and VU-mode trigger access is specified by the distinct [Sucschrind](#) and [Sutcfg](#) extensions. Those extensions are not part of Smtdeleg or Sstcfg.



This specification treats S/HS-mode software as trusted to manage trigger state assigned to lower-privilege VS-mode, VU-mode, and U-mode contexts. The manager accesses defined in this chapter and in [Sutcfg](#) do not protect that lower-privilege trigger state from S/HS-mode software.

4.1. Sstcfg Scope

Sstcfg includes:

- The **stselect** CSR for supervisor trigger selection.
- Trigger access behavior via **siselect/sireg***, including VS-mode substitution to **vsiselect/vsireg*** when V=1.
- The state access modifications applied when triggers are accessed through the indirect interface.

Smtdeleg encompasses the Sstcfg functionality, and additionally includes:

- The **mstateen0.TR** and **hstateen0.TR** bit definitions for access control.
- The machine-level specification of delegation and virtualization behavior.

4.2. Supervisor Trigger Select (**stselect**)

Table 11. Supervisor Trigger Select CSR

Number	Privilege	Width	Name	Description
address TBD	SRW	XLEN	stselect	Supervisor trigger select

stselect provides S/HS-mode read/write access to the trigger selection index, equivalent to **tselect** for M-mode. **stselect** and **tselect** access the same underlying state: writing **stselect** updates the trigger index that **tselect** reads, and vice versa. When [Sutcfg](#) is implemented, **utselect** also accesses this same shared state.

Software must save and restore this selection state when switching trigger access between M-mode, S/HS-mode, VS-mode, and U/VU-mode. M-mode software must not assume that **tselect** retains its value across

delegated accesses.

stselect is WARL and follows the same legal-value conversion as Sdtrig **tselect**. Writes of values greater than or equal to the number of supported triggers may result in a different value in **stselect** than what was written, or may select a trigger where **tdata1**.TYPE=0. To verify that a written index is valid, software can read back **stselect** and check that it holds the written value, then read **sireg** while **siselect** holds the trigger access value (or **vsireg** while **vsiselect** holds the trigger access value when V=1) and check that TYPE is non-zero.

When V=1, **stselect** is used by both S/HS-mode and VS-mode. Software must save and restore **stselect** when switching between S/HS-mode and VS-mode contexts.

4.3. Supervisor and Virtual Supervisor Trigger Access



The **siselect** value for trigger access is to be allocated from the Sscsrind select address space. When [Sutcfg](#) is implemented, the same value is used for **uiselect** trigger access. Throughout this document, this value is referred to as the trigger access value.

While **siselect** holds the trigger access value, the **sireg*** registers provide supervisor read/write access to register state associated with the trigger selected by **stselect**, the same state accessed by M-mode CSRs **tdata*** and **tinfo**. The register mapping is shown below.

Table 12. Indirect Trigger Register Mappings When **siselect** Holds the Trigger Access Value

Indirect CSR	State Accessed
sireg	Same as tdata1
sireg2	Same as tdata2
sireg3	Same as tdata3
sireg4	Same as tinfo (read-only; writes ignored), with type bits restricted as specified in Section 4.4.1



This method requires one new CSR (**stselect**) and one **siselect** index. Allocating a range of **siselect** values to select the trigger, bypassing **tselect**, would add no new CSRs but would cap the number of triggers accessible in S/HS-mode. **tselect** supports up to 2^{MXLEN} triggers, while a dedicated **siselect** range would likely cover only a practical subset.

While the privilege mode is M-mode or S/HS-mode and **siselect** holds the trigger access value, illegal-instruction exceptions are raised for attempts to access **sireg5** or **sireg6**.

When V=1, attempts from VS-mode to access **sireg5** or **sireg6** (really **vsireg5** or **vsireg6**) while **vsiselect** holds the trigger access value raise an illegal-instruction exception.

When HS-mode accesses **vsireg5** or **vsireg6** while **vsiselect** holds the trigger access value, an illegal-instruction exception is raised.

4.3.1. Virtualization

When V=1, accesses from VS-mode to **siselect** and **sireg*** are substituted by **vsiselect** and **vsireg***, as specified by Sscsrind. Field relocations that apply to VS-mode accesses are specified in [Section 4.4](#). Since **stselect** is shared between S/HS-mode and VS-mode, software must save and restore **stselect** when switching between S/HS-mode and VS-mode contexts.

When HS-mode accesses **vsiselect** and **vsireg*** directly, the accesses use the VS-mode field modifications

and type constraints in this chapter. These accesses provide HS-mode management of VS-mode and VU-mode trigger context as specified in [Section 4.4.3](#).

4.4. Supervisor Trigger State Access Modifications

sireg* and **vsireg*** provide access to a subset of the fields in the native trigger registers, and in some cases fields are relocated. The field modifications that apply when trigger registers are accessed via **sireg*** or **vsireg*** are documented below.

Table 13. Delegated Trigger CSR Field Modifications

Sdtrig Register(s)	sireg* Field Modifications	vsireg* Field Modifications
mcontrol6, icount, etrigger, itrigger (accessed by sireg / vsireg)	M is read-only 0	M is read-only 0 The S field maps to the underlying VS field in tdata1 The U field maps to the underlying VU field in tdata1 VS and VU are read-only 0
textra32, textra64 (accessed by sireg3 / vsireg3)	none	MHVALUE and MHSELECT are read-only 0



The bit positions for the M, S, U, VS, and VU fields vary based on the trigger type specified by **tdata1.TYPE**; see *The RISC-V Debug Specification* [1], Chapter 5.7.

4.4.1. Type Delegation Constraints

The following constraints apply to permitted accesses through **sireg*** and **vsireg***.

The trigger types supported through **stselect** with **siselect/sireg***, and through VS-mode substituted accesses (**vsiselect/vsireg***), are **mcontrol6, icount, itrigger, etrigger**, and disabled (Sdtrig type 15). When the selected trigger's **tdata1.TYPE** is any other value, including **mcontrol, tmexttrigger, legacy, custom**, and reserved types, accesses through **sireg, sireg2, sireg3**, and **sireg4** are read-only 0 and writes are ignored. The same applies to **vsireg** through **vsireg4** when V=1 or when they are accessed directly from HS-mode as specified in [Section 4.4.3](#).

When **tdata1.TYPE** is a supported type, **sireg4** and **vsireg4** (including direct HS-mode manager access) read **tinfo** with bits corresponding to types not supported through this interface read-only 0. Writes to **tdata1.TYPE** through **sireg** or **vsireg** are WARL and may only select a type supported through this interface.

Table 14. Type-Specific Constraints for Delegated Trigger Access

Trigger type	Delegated register-interface constraints
itrigger	Programming access is allowed only for interrupt causes that a handler at the S/HS view or the VS view can observe. Through sireg* , sireg2 bit <i>i</i> is writable if midelg[i] =1. Through vsireg* , writability follows the interrupt cause number reported to the VS-mode handler (vscause), after the standard virtual-interrupt translation described in the following NOTE. Bits that are not permitted are read-only 0 and writes are ignored. The nmi field in tdata1 (accessed through sireg / vsireg) is read-only 0; NMI is not delegated by midelg or hideleg .

Trigger type	Delegated register-interface constraints
etrigger	Programming access is allowed only for exception causes permitted in the S/HS view or the VS view. Through sireg* , sireg2 bit <i>i</i> is writable if medeleg [<i>i</i>]=1. Through vsireg* , vsireg2 bit <i>i</i> is writable if hedeleg [<i>i</i>]=1. Exception cause numbers are not translated. Bits that are not permitted are read-only 0 and writes are ignored.



mcontrol is deprecated and is not supported through this interface. Use **mcontrol6** for address/control triggers.



*Sdtrig interprets each **itrigger tdata2** bit as the interrupt number in the mode in which the trap handler executes; virtualized interrupt numbers are not the same in every mode. When a virtual supervisor interrupt is delegated to VS-mode, the Hypervisor extension translates the cause written to **vscause**: code 10 becomes 9, code 6 becomes 5, and code 2 becomes 1. VS-mode programming of **vsireg2** therefore permits bit 9 when **hedeleg**[10]=1, bit 5 when **hedeleg**[6]=1, and bit 1 when **hedeleg**[2]=1. A same-numbered test of **hedeleg**[*i*] for bit *i* would reject these VS-visible causes (**hedeleg** bits 9, 5, and 1 are read-only 0) and would permit the untranslated codes 10, 6, and 2, which a VS-mode handler does not observe. For interrupt causes 16 and above, VS-mode uses the translated cause number if the Hypervisor extension translates that cause when the interrupt is delegated to VS-mode; otherwise it uses the **hedeleg** bit index.*

For **etrigger**, exception codes are not translated, so VS-mode uses **hedeleg**[*i*] for **vsireg2** bit *i*. Trap delegation is hierarchical: an interrupt or exception delegated to VS-mode must first be delegated to S/HS-mode. S/HS-mode save and restore of guest **etrigger** state therefore uses the same **medeleg**[*i*] permission as for S/HS-mode's own causes. Guest **itrigger** state uses the VS-visible cause bits 1, 5, and 9 for the standard virtual supervisor interrupts. For these standard virtual interrupts, the corresponding host delegation conditions are **medeleg**[10], **medeleg**[6], and **medeleg**[2], respectively, together with **hedeleg**[10], **hedeleg**[6], and **hedeleg**[2], respectively. The **medeleg** bits for these virtual interrupts are read-only 1 when H is implemented and delegate them to HS-mode, while the **hedeleg** bits delegate the virtual interrupts to VS-mode.

4.4.2. DMODE Interaction

When **tdata1.DMODE** = 1 for the trigger selected by **stselect**, the trigger is reserved for Debug Mode use. If the hart is not in Debug Mode, accesses through **sireg**, **sireg2**, **sireg3**, and **sireg4** by S/HS-mode software while **siselect** holds the trigger access value are read-only 0. When V=1, accesses through **vsireg**, **vsireg2**, **vsireg3**, and **vsireg4** by VS-mode software while **vsiselect** holds the trigger access value are likewise read-only 0. Direct HS-mode manager accesses through **vsireg*** are governed by [Section 4.4.3](#). If the hart is in Debug Mode, those accesses are not forced to read-only 0; they follow the field modifications in this section together with the base Sdtrig **DMODE** rules.

4.4.3. HS-mode Management of VS/VU Trigger Context

When HS-mode accesses **vsiselect** and **vsireg*** directly while **vsiselect** holds the trigger access value, the access is a manager access to trigger state assigned to VS-mode or VU-mode contexts. If the hart is not in Debug Mode and the selected trigger has **tdata1.DMODE**=1, **vsireg**, **vsireg2**, **vsireg3**, and **vsireg4** are not forced to read-only 0 for this HS-mode manager access. HS-mode software may therefore save and restore the **DMODE** field and the other fields exposed by the VS-mode view.

For an HS-mode access to **vsireg** while **vsiselect** holds the trigger access value, **DMODE** is read/write. An HS-mode write through **vsireg** of a disabled-trigger encoding (**TYPE**=15, **DMODE**=0, and **ACTION**=0) is permitted even when the selected trigger previously had **DMODE**=1 and atomically disables that trigger. Other HS-mode writes through **vsireg** are permitted to update **DMODE** and **ACTION** in the same **tdata1** write.

If an HS-mode write through **vsireg** would result in **DMODE=0** and **ACTION=1**, the implementation must set **ACTION** to 0. All other field modifications, type constraints, and type-specific constraints in this chapter continue to apply. VS-mode software remains subject to the read-only-0 rule in [Section 4.4.2](#), and this manager access does not provide HS-mode with native access to fields masked by the VS-mode view.

When switching between VS-mode or VU-mode contexts, HS-mode software should save **stselect**, **vsiselect**, and the exposed **vsireg** through **vsireg4** state. It should first write **vsireg** with the disabled-trigger encoding (**TYPE=15**, **DMODE=0**, and **ACTION=0**), which is permitted even when the selected trigger previously had **DMODE=1**, so the selected trigger cannot match while its state is being changed. The mode-enable fields hidden by the VS-mode view must already be clear by cooperative trigger allocation. HS-mode software can then write **vsireg2** and **vsireg3**, followed by the final **vsireg** value; the final write may restore **DMODE=1** and **ACTION=1** together when that was the saved configuration. It should restore **stselect** and **vsiselect** after the context switch.

These HS-mode manager accesses are architectural accesses and do not require an SBI. When **Smstateen** is implemented, they remain subject to **mstateen0.CSRIND** and **mstateen0.TR**. **hstateen0.TR** controls VS-mode and VU-mode software access; it does not restrict the HS-mode manager.



*When the hart is not in Debug Mode and **sireg** through **sireg4** all read as zero for a trigger index held by **stselect**, or, when **V=1**, **vsireg** through **vsireg4** all read as zero, the selected slot is unavailable through this interface. The slot may be reserved by Debug Mode (**DMODE=1**), may have an unsupported **tdata1.TYPE**, or may have **tdata1.TYPE=0**. Software should try the next trigger index. An all-zero read of aliases 1–4 does not distinguish these cases. **Sdtrig chain** WARL on a visible trigger still depends on a neighboring trigger's **DMODE**; this interface does not redefine that WARL. An external debugger whose [debug access privilege](#) is S/HS-mode or VS-mode uses the same interface from Debug Mode, where **DMODE=1** is visible and writable, so it can program **ACTION=1** in accordance with **Sdtrig**.*

4.5. Supervisor and Virtual Supervisor Trigger State Access Control

When **Smstateen** is implemented, access is checked in the following order. When **V=1** and the privilege mode is VS, accesses to **siselect** and **sireg*** are substituted by **vsiselect** and **vsireg*** as specified by **Sscsrind**. The following checks apply to those effective CSRs.

1. If the privilege mode is less privileged than M-mode and **mstateen0.TR=0**, attempts to access **stselect**, or to access **sireg**, **sireg2**, **sireg3**, or **sireg4** while **siselect** holds the trigger access value, or, from HS-mode, to access **vsireg**, **vsireg2**, **vsireg3**, or **vsireg4** while **vsiselect** holds the trigger access value, raise an illegal-instruction exception.
2. Otherwise, if **V=1** and the privilege mode is VS or VU and **hstateen0.TR=0**, attempts to access **stselect**, or to access **sireg**, **sireg2**, **sireg3**, or **sireg4** while **siselect** holds the trigger access value, raise a virtual-instruction exception.
3. Otherwise, if the privilege mode is M-mode, S/HS-mode, or **V=1** and the privilege mode is VS, accesses to **stselect** are permitted, and accesses to **sireg**, **sireg2**, **sireg3**, and **sireg4** while **siselect** holds the trigger access value are permitted. HS-mode accesses to **vsireg**, **vsireg2**, **vsireg3**, and **vsireg4** while **vsiselect** holds the trigger access value are also permitted.

M-mode accesses through **stselect** and **sireg*** use the S/HS-view field modifications and type constraints in this chapter; they do not use the native **tdata*** encoding.

hstateen0.TR is read-only 0 when **mstateen0.TR=0**.

The TR bit is bit 3 in **mstateen0** and **hstateen0**. [Sutcfg](#) defines **sstateen0.TR** to control U-mode access to delegated triggers. When [Sutcfg](#) is not implemented, **sstateen0.TR** remains read-only 0.



See the *Sscsrind* specification [5] for how bit 60 (*CSRIND*) in **mstateen0** and **hstateen0** can also restrict access to **siselect** and **sireg*** from privilege modes less privileged than M-mode, including VS-mode when *V*=1. The *CSRIND* bit provides blanket control over indirect CSR infrastructure, while *TR* provides trigger-specific delegation control.



Smtdeleg uses one *TR* bit per privilege level to control delegated trigger access. If per-trigger delegation is needed in the future, new delegation registers (for example **tdelegX**/**htdelegX**) could be introduced.

4.6. Access to **tcontrol**

The **tcontrol** CSR is not accessible through the delegated indirect interface. **tcontrol** remains M-mode only.

When a delegated trigger with *ACTION*=0 fires in S/HS-mode or VS-mode, a breakpoint exception is taken through the normal privilege trap mechanism (**scause/stval** for S/HS-mode, **vscause/vstval** for VS-mode). *ACTION*=1 follows **Sdtrig** and enters Debug Mode, except that it does not match or fire while external debug is disallowed for the hart's current privilege mode, as specified in Section 3.1. Other *ACTION* values retain the behavior defined by **Sdtrig**.



tcontrol.MTE and **tcontrol.MPTE** affect only triggers with *ACTION*=0 while the hart is in M-mode, as specified by **Sdtrig**. They do not disable triggers that match or fire in a less-privileged mode, and they do not affect *ACTION*=1. Delegated *ACTION*=0 triggers follow **Sdtrig**'s native-trigger match and reentrancy rules. Because **tcontrol** is not accessible from S-mode, it does not provide a less-privileged analogue of the M-mode handler-reentrancy control.

4.7. Debugger Access from S/HS-mode and VS-mode

When **Smsdbgsec** is implemented, **mdbggen**=0, and **SEDBGGEN**=1, an external debugger whose [debug access privilege](#) is S/HS-mode can access triggers through the **stselect** + **siselect/sireg*** interface, provided that **mstateen0.TR**=1.

When **Smvsdbgsec** is implemented, **mdbggen**=0, **SEDBGGEN**=0, and **VSEDBGGEN**=1, an external debugger whose debug access privilege is VS-mode uses the same interface with the *Sscsrind* substitution to **vsiselect/vsireg***, provided that **mstateen0.TR**=1 and **hstateen0.TR**=1.



This access path is one of the primary motivations for **Smtdeleg**: without it, an external debugger restricted to S/HS-mode or VS-mode privilege has no path to trigger state. For an external debugger restricted to U-mode or VU-mode privilege, [Sutcfg](#) provides the corresponding access path via **utselect** and **uiselect/uireg***.

4.8. Trigger Allocation

Smtdeleg uses **Smstateen** bits for delegation control. When **mstateen0.TR**=1, S/HS-mode can access delegated trigger state through **stselect** and **siselect/sireg*** (subject to *DMODE* restrictions and type-specific constraints). When *V*=1 and **hstateen0.TR**=1, VS-mode uses the same selection state with the *Sscsrind* substitution to **vsiselect/vsireg***. HS-mode can also use direct **vsiselect/vsireg*** accesses to manage VS-mode and VU-mode trigger context as specified in Section 4.4.3.

DMODE takes priority over delegated S/HS-mode and VS-mode access. When **tdata1.DMODE**=1:

- Ordinary S/HS-mode and VS-mode software can write the trigger's **tdata*** fields only under the base

Sdtrig rules. When `mdbggen=0`, M-mode writes to `tdata1.DMODE` follow [\[mdbggen-dmode\]](#). HS-mode manager accesses to VS-mode and VU-mode trigger state are the exception defined in [Section 4.4.3](#) and may manage `DMODE=1` state.

- S/HS-mode and VS-mode accesses through `sireg*/vsireg*` follow [Section 4.4.2](#): they are read-only 0 if the hart is not in Debug Mode, and are not forced to read-only 0 if the hart is in Debug Mode. Direct HS-mode manager accesses through `vsireg*` are governed by [Section 4.4.3](#).

Higher-privilege software can reconfigure triggers with `DMODE=0` through its own interface, overriding lower-privilege configuration. A `DMODE=1` trigger in the S/HS view is not reclaimed through `siselect/sireg*` while the hart is not in Debug Mode. HS-mode can manage `DMODE=1` trigger state assigned to VS-mode and VU-mode contexts through the manager access in [Section 4.4.3](#). Debug Mode can reclaim a trigger through the corresponding delegated interface. When `mdbggen=0`, M-mode can park or restore native trigger state as specified in [\[mdbggen-dmode\]](#). Less-privileged software that needs a `DMODE=1` slot in the S/HS view released requests M-mode assistance.

Software must manage trigger allocation cooperatively. TR enables delegated trigger access but does not identify per-trigger ownership. HS-mode manager access to the VS/VU view requires `mstateen0.CSRIND` and `mstateen0.TR`; `hstateen0.TR` controls VS-mode and VU-mode software access and does not restrict the HS-mode manager. The manager access does not provide access to the S/HS view's `DMODE=1` slots. There is no per-trigger hardware allocation. Before setting the applicable TR bit, higher-privilege software should clear its own mode-enable bits on any trigger it is exposing.

- M-mode allocates triggers between its own use and S/HS-mode delegation.
- M-mode firmware sets `mstateen0.TR=1` once it is prepared to delegate trigger access.
- S/HS-mode software allocates triggers between its own use and VS delegation.
- When [Sutcfg](#) is implemented, S/HS-mode software allocates triggers between its own use and U-mode delegation.

4.9. Dependencies

Smtdeleg depends on the following extensions:

- Sdtrig — Base trigger extension defining trigger CSRs and types.
- Sscsrind / Smcsrind — Indirect CSR interface framework [\[5\]](#).
- Smstateen / Ssstateen — State enable framework for access control.
- H extension (optional) — Required for virtualization controls via `hstateen0`.

Sstcfg requires Smtdeleg. The dependencies listed above for Smtdeleg therefore also apply to Sstcfg.

U-mode and VU-mode trigger access is provided by [Sutcfg](#), which requires Sucsrind and, when S-mode is implemented, also requires Smtdeleg and Sstcfg.

4.10. Summary of New State

Table 15. New CSRs Introduced by Sstcfg

CSR	Address	Privilege	Notes
<code>stselect</code>	address TBD	SRW	Supervisor trigger select

Table 16. New Smstateen Bits Introduced by Smtdeleg

Register	Bit	Purpose
mstateen0	3 (TR)	Controls access to delegated triggers below M-mode
hstateen0	3 (TR)	Controls delegated trigger access in VS-mode when V=1



The `sstateen0.TR` bit for U-mode delegation is defined by [Sutcfg](#).

Chapter 5. Sucsrrind (ISA Extension)

This chapter specifies the Sucsrrind ISA extension, which extends the indirect CSR access mechanism defined by Smcsrrind [5] to U-mode. Sucsrrind is not a member of the Secure Debug or Secure Trace families.

Sucsrrind provides U-mode with its own set of indirect register select and alias CSRs. This enables extensions to expose register arrays to U-mode software without consuming large portions of the limited user-level CSR address space, mirroring the benefits Sscsrrind provides at supervisor level.



The Sscsrrind specification reserves CSR address space for this extension: CSR address space is reserved for a possible future Sucsrrind extension that extends indirect CSR access to user mode.

Sucsrrind requires Smcsrrind. When S-mode is implemented, Sucsrrind also requires Sscsrrind. Smcsrrind encompasses all added CSRs and all behavior modifications for a hart, over all privilege levels.

5.1. User-level CSRs

Table 17. User Indirect CSR Access CSRs

Number	Privilege	Width	Name	Description
address TBD	URW	XLEN	uiselect	User indirect register select
address TBD	URW	XLEN	uireg	User indirect register alias
address TBD	URW	XLEN	uireg2	User indirect register alias 2
address TBD	URW	XLEN	uireg3	User indirect register alias 3
address TBD	URW	XLEN	uireg4	User indirect register alias 4
address TBD	URW	XLEN	uireg5	User indirect register alias 5
address TBD	URW	XLEN	uireg6	User indirect register alias 6



*The **uireg*** CSR numbers are not consecutive because one address in the allocated range is reserved, consistent with the gap in the Sscsrrind supervisor-level and machine-level indirect alias CSR numbering. Final CSR addresses are allocated by ARC.*

The value of **uiselect** determines which register is accessed upon read or write of each of the user indirect alias CSRs (**uireg***). **uiselect** value ranges are allocated to dependent extensions, which specify the register state accessible via each **uireg*** register, for each **uiselect** value. **uiselect** is a WARL register.

The **uiselect** register will support the value range 0..0xFFFF at a minimum. A future extension may define a value range outside of this minimum range. Only if such an extension is implemented will **uiselect** be required to support larger values.



*Requiring a range of 0-0xFFFF for **uiselect**, even though most or all of the space may be reserved or inaccessible, permits S-mode to emulate indirectly accessed registers in this implemented range, including registers that may be standardized in the future.*

Values of **uiselect** with the most-significant bit set (bit XLEN-1 = 1) are designated only for custom use. Values of **uiselect** with the most-significant bit clear are designated only for standard use and are reserved until allocated to a standard architecture extension. If XLEN is changed, the most-significant bit of **uiselect** moves to the new position, retaining its value from before.

The behavior upon accessing **uireg*** from M-mode, S-mode, or U-mode, while **uiselect** holds a value that

is not implemented at user level, is UNSPECIFIED.



It is expected that implementations will typically raise an illegal-instruction exception for such accesses, so that, for example, they can be identified as software bugs.

Attempts to access **uireg*** while **uiselect** holds a number in an allocated and implemented range results in a specific behavior that, for each combination of **uiselect** and **uireg***, is defined by the extension to which the **uiselect** value is allocated.



*Ordinarily, each **uireg*** will access register state, access read-only O state, or raise an illegal-instruction exception.*

User-level, supervisor-level, and machine-level select values are separate address spaces from each other. An extension may define user-level and supervisor-level CSRs with the same select value as partial or full aliases, typically for state that can be delegated from supervisor-level to user-level.

The widths of **uiselect** and **uireg*** are always the current XLEN.

5.2. Virtualization

Sucsrind does not define separate virtual-user **vu*** indirect CSRs.

When V=1, accesses from VS-mode or VU-mode to **uiselect** and **uireg*** use the same CSR interface and semantics as U-mode accesses but applied to VU-mode. Higher privilege modes are expected to swap the content of these registers as needed.

5.3. Access Control by the State-Enable CSRs

If extension Smstateen is implemented together with Sucsrind, bit 60 (CSRIND) of state-enable register **mstateen0** controls access to **uiselect** and **uireg***, in addition to the **siselect**, **sireg***, **vsiselect**, and **vsireg*** CSRs it already controls under Sscsrind [6]. When **mstateen0**[60]=0, an attempt to access **uiselect** or **uireg*** from a privilege mode less privileged than M-mode results in an illegal-instruction exception.

If the hypervisor extension is implemented, **hstateen0**[60] (CSRIND) also controls accesses from VS-mode or VU-mode to **uiselect** and **uireg***. When **hstateen0**[60]=0 and **mstateen0**[60]=1, all attempts from VS-mode or VU-mode to access **uiselect** or **uireg*** raise a virtual-instruction exception.



*Individual extensions that use **uiselect**/**uireg*** to access specific register state may define additional **sstateen** bits to provide S-mode with fine-grained control over U-mode access to that state. [Sutcfg](#) defines **sstateen0.TR** to control U-mode access to trigger registers accessed through **uireg***.*

5.4. Dependencies

Sucsrind depends on the following extensions:

- Smcsrind — Machine-level indirect CSR access framework [5].
- Sscsrind (required when S-mode is implemented) — Supervisor-level indirect CSR access framework.
- S-mode (optional) — Required for **sstateen0** access control and for Sscsrind. In implementations without S-mode, **mstateen0** alone controls U-mode access to **uiselect**/**uireg***. When Smstateen is not implemented, all indirect CSR state is accessible as defined by this extension.

- H extension (optional) — Required for VS/VU-mode virtualization support.

5.5. Summary of New State

Table 18. New CSRs Introduced by Sucsrind

CSR	Address	Privilege	Notes
uiselect	address TBD	URW	User indirect register select
uired	address TBD	URW	User indirect register alias
uired2	address TBD	URW	User indirect register alias 2
uired3	address TBD	URW	User indirect register alias 3
uired4	address TBD	URW	User indirect register alias 4
uired5	address TBD	URW	User indirect register alias 5
uired6	address TBD	URW	User indirect register alias 6

Chapter 6. Sutcfg (ISA Extension)

This chapter specifies the Sutcfg ISA extension, which provides U-mode and VU-mode access to Sdtrig [1] triggers using the user-level indirect CSR interface defined by [Sucsrind](#). Sutcfg is not a member of the Secure Debug or Secure Trace families.

When S-mode is implemented, U-mode access requires **mstateen0**.TR=1 and **sstateen0**.TR=1. When V=1, VU-mode access additionally requires **hstateen0**.TR=1 and virtual **sstateen0**.TR=1. When S-mode is not implemented, **mstateen0**.TR directly controls U-mode access.

Sutcfg enables:

- U-mode and VU-mode self-hosted debuggers to access triggers using **utselect**, **uiselect**, and **uireg***.
- An external debugger to access triggers through **utselect**, **uiselect**, and **uireg*** when a Secure Debug extension is implemented and Debug Mode has less than M-mode privilege.

Sutcfg is a distinct extension. Its implementation requires Sucsrind. When S-mode is implemented, Sutcfg also requires [Smtdeleg](#) and [Sstcfg](#).

6.1. User Trigger Select (**utselect**)

Table 19. User Trigger Select CSR

Number	Privilege	Width	Name	Description
address TBD	URW	XLEN	utselect	User trigger select

utselect provides U-mode read/write access to the trigger selection index. **utselect** and **tselect** access the same underlying state: writing **utselect** updates the trigger index that **tselect** reads, and vice versa. When S-mode is implemented, **stselect** also accesses this same shared state.

Software must save and restore this selection state when switching trigger access between M-mode, S/HS-mode, VS-mode, and U/VU-mode. Higher-privilege software must not assume that **tselect** or **stselect** retains its value across delegated U-mode accesses.

utselect is WARL and follows the same legal-value conversion as Sdtrig **tselect**. Writes of values greater than or equal to the number of supported triggers may result in a different value in **utselect** than what was written, or may select a trigger where **tdata1**.TYPE=0. To verify that a written index is valid, software can read back **utselect** and check that it holds the written value, then read **uireg** while **uiselect** holds the trigger access value and check that TYPE is non-zero.

6.2. User Trigger Access



The **uiselect** value for trigger access is to be allocated from the Sucsrind select address space. When [Smtdeleg](#) and [Sstcfg](#) are implemented, the same value is used for **siselect** trigger access. Throughout this document, this value is referred to as the trigger access value. See [Section 4.3](#).

While **uiselect** holds the trigger access value, the **uireg*** registers provide user-mode read/write access to register state associated with the trigger selected by **utselect**, the same state accessed by M-mode CSRs **tdata*** and **tinfo**. The register mapping is shown below.

Table 20. Indirect Trigger Register Mappings When **uiselect** Holds the Trigger Access Value

Indirect CSR	State Accessed
<code>uireg</code>	Same as <code>tdata1</code>
<code>uireg2</code>	Same as <code>tdata2</code>
<code>uireg3</code>	Same as <code>tdata3</code>
<code>uireg4</code>	Same as <code>tinfo</code> (read-only; writes ignored), with type bits restricted as specified in Section 6.3.1

While `uiselect` holds the trigger access value, attempts to access `uireg5` or `uireg6` raise an illegal-instruction exception.

6.2.1. Virtualization

When `V=1`, accesses from VS-mode or VU-mode to `uiselect` and `uireg*` follow [Sucsrint](#) virtualization rules. Field relocations that apply to VS-mode or VU-mode accesses are specified in [Section 6.3](#).

6.3. User Trigger State Access Modifications

`uireg*` provides access to a subset of the fields in the native trigger registers, and in some cases fields are relocated. The field modifications that apply when trigger registers are accessed via `uireg*` are documented below.

Table 21. U-mode and VS/VU-mode Trigger CSR Field Modifications

Sdtrig Register(s)	<code>uireg*</code> Field Modifications (U-mode)	<code>uireg*</code> Field Modifications (VS/VU-mode)
<code>mcontrol6</code> , <code>icount</code> (accessed by <code>uireg</code>)	M, S, VS, and VU are read-only 0	M, S, VS, and VU are read-only 0 The U field maps to the underlying VU field in <code>tdata1</code>
<code>textra32</code> , <code>textra64</code> (accessed by <code>uireg3</code>)	MHVALUE, MHSELECT, SVALUE, SSELECT, and SBYTEMASK are read-only 0	MHVALUE, MHSELECT, SVALUE, SSELECT, and SBYTEMASK are read-only 0



The bit positions for the M, S, U, VS, and VU fields vary based on the trigger type specified by `tdata1.TYPE`; see [The RISC-V Debug Specification \[1\]](#), Chapter 5.7.

6.3.1. Type Delegation Constraints

The trigger types supported through `utselect`, `uiselect`, and `uireg*` are `mcontrol6`, `icount`, and disabled (Sdtrig type 15). When the selected trigger’s `tdata1.TYPE` is any other value, including `mcontrol`, `itrigger`, `etrigger`, `tmexttrigger`, legacy, custom, and reserved types, accesses through `uireg`, `uireg2`, `uireg3`, and `uireg4` are read-only 0 and writes are ignored.

When `tdata1.TYPE` is a supported type, `uireg4` reads `tinfo` with bits corresponding to types not supported through this interface read-only 0. Writes to `tdata1.TYPE` through `uireg` are WARL and may only select a type supported through this interface.



`mcontrol` is deprecated and is not supported through this interface. Use `mcontrol6` for address/control triggers.

6.3.2. DMODE Interaction

When `tdata1.DMODE = 1` for the trigger selected by `utselect`, the trigger is reserved for Debug Mode use. If the hart is not in Debug Mode, accesses through `uired`, `uired2`, `uired3`, and `uired4` by VS-mode, U-mode, or VU-mode software while `uiselect` holds the trigger access value are read-only 0. S/HS-mode accesses through these aliases, when permitted by the access-control rules below, are manager accesses to U-mode trigger state and are not forced to read-only 0. If the hart is in Debug Mode, VS-mode, U-mode, and VU-mode accesses are not forced to read-only 0; they follow the field modifications in this section together with the base Sdtrig **DMODE** rules.

For an S/HS-mode access to `uired` while `uiselect` holds the trigger access value, **DMODE** is read/write. An S/HS-mode write through `uired` of a disabled-trigger encoding (**TYPE**=15, **DMODE**=0, and **ACTION**=0) is permitted even when the selected trigger previously had **DMODE**=1 and atomically disables that trigger. Other S/HS-mode writes through `uired` are permitted to update **DMODE** and **ACTION** in the same `tdata1` write. If an S/HS-mode write through `uired` would result in **DMODE**=0 and **ACTION**=1, the implementation must set **ACTION** to 0. All other field modifications, type constraints, and type-specific constraints in this chapter continue to apply. VS-mode, U-mode, and VU-mode software remain subject to the read-only-0 rule above.

When switching between U-mode application contexts, S/HS-mode software should save `utselect`, `uiselect`, and the exposed `uired` through `uired4` state. It should first write `uired` with the disabled-trigger encoding (**TYPE**=15, **DMODE**=0, and **ACTION**=0), which is permitted even when the selected trigger previously had **DMODE**=1, so the selected trigger cannot match while its state is being changed. The mode-enable fields hidden by the U-mode view must already be clear by cooperative trigger allocation. S/HS-mode software can then clear the applicable U-mode enable field, write `uired2` and `uired3`, followed by the final `uired` value; the final write may restore **DMODE**=1 and **ACTION**=1 together when that was the saved configuration. It should restore `utselect` and `uiselect` after the context switch. VU-mode context is managed through the HS-mode `vsired*` path specified in [Section 4.4.3](#).



*When the hart is not in Debug Mode and `uired` through `uired4` all read as zero for a trigger index held by `utselect`, VS-mode, U-mode, or VU-mode software cannot use the selected slot through this interface. The slot may be reserved by Debug Mode (**DMODE**=1), may have an unsupported `tdata1.TYPE`, or may have `tdata1.TYPE`=0. Software should try the next trigger index. An all-zero read of aliases 1–4 does not distinguish these cases. S/HS-mode manager accesses are governed separately above. Sdtrig **chain** WARL on a visible trigger still depends on a neighboring trigger's **DMODE**; this interface does not redefine that WARL. An external debugger whose [debug access privilege](#) is U-mode or VU-mode uses the same interface from Debug Mode, where **DMODE**=1 is visible and writable, so it can program **ACTION**=1 in accordance with Sdtrig.*

6.4. User Trigger State Access Control

When S-mode is implemented, U-mode access requires `mstateen0.TR=1` and `sstateen0.TR=1`. When V=1, VU-mode access additionally requires `hstateen0.TR=1`. The access control is checked in the following order:

1. If the privilege mode is less privileged than M-mode and `mstateen0.TR=0`, attempts to access `utselect`, or to access `uired`, `uired2`, `uired3`, or `uired4` while `uiselect` holds the trigger access value, raise an illegal-instruction exception.
2. Otherwise, if V=1 and the privilege mode is VS or VU and `hstateen0.TR=0`, attempts to access `utselect`, or to access `uired`, `uired2`, `uired3`, or `uired4` while `uiselect` holds the trigger access value, raise a virtual-instruction exception.
3. Otherwise, if V=0 and the privilege mode is U and `sstateen0.TR=0`, attempts to access `utselect`, or to

access **uireg**, **uireg2**, **uireg3**, or **uireg4** while **uiselect** holds the trigger access value, raise an illegal-instruction exception.

- Otherwise, if $V=1$ and the privilege mode is VU and virtual **sstateen0**.TR=0, attempts to access **utselect**, or to access **uireg**, **uireg2**, **uireg3**, or **uireg4** while **uiselect** holds the trigger access value, raise a virtual-instruction exception.

When S-mode is implemented, VS-mode accesses through this interface use the VU-mode field view and are controlled by **hstateen0**.TR; U-mode accesses are controlled by **sstateen0**.TR; and VU-mode accesses are controlled by both **hstateen0**.TR and virtual **sstateen0**.TR. S/HS-mode manager accesses use **mstateen0**.CSRIND and **mstateen0**.TR and are not restricted by these lower-level state-enable bits.

When S-mode is not implemented, **sstateen0** does not exist. In this case, **mstateen0**.TR alone controls U-mode access to **utselect** and to **uireg** through **uireg4** while **uiselect** holds the trigger access value. An illegal-instruction exception is raised for those accesses when **mstateen0**.TR=0.

M-mode accesses to **utselect** and to **uireg** through **uireg4** while **uiselect** holds the trigger access value are not restricted by TR and are permitted. When S-mode is implemented and **mstateen0**.TR=1, S/HS-mode manager accesses to those CSRs are likewise permitted. Those accesses use the U-mode field modifications and type constraints in this chapter, including the S/HS-mode manager access to **Dmode** described above.

S/HS-mode manager accesses are architectural accesses and do not require an SBI. When **Smstateen** is implemented, they remain subject to **mstateen0**.CSRIND and **mstateen0**.TR. **sstateen0**.TR and **hstateen0**.TR control VS-mode, U-mode, and VU-mode software access; they do not restrict the S/HS-mode manager.

When **mstateen0**.TR=0, **sstateen0**.TR is read-only 0. When $V=1$ and **hstateen0**.TR=0, virtual **sstateen0**.TR is read-only 0.

The TR bit is bit 3 in **sstateen0**.



See the *Sscsrind* and *Sucsrind* specifications [5] for how bit 60 (CSRIND) in **mstateen0** and **hstateen0** can also restrict access to **uiselect** and **uireg*** from privilege modes less privileged than M-mode, including VU-mode when $V=1$. The CSRIND bit provides blanket control over indirect CSR infrastructure, while TR provides trigger-specific delegation control.

6.5. Access to **tcontrol**

The **tcontrol** CSR is not accessible through the delegated indirect interface. **tcontrol** remains M-mode only.

When a delegated trigger with ACTION=0 fires in U-mode or VU-mode, a breakpoint exception is taken through the normal privilege trap mechanism. ACTION=1 follows **Sdtrig** and enters Debug Mode, except that it does not match or fire while external debug is disallowed for the hart's current privilege mode, as specified in [Section 3.1](#). Other ACTION values retain the behavior defined by **Sdtrig**.



tcontrol.MTE and **tcontrol**.MPTE affect only triggers with ACTION=0 while the hart is in M-mode, as specified by **Sdtrig**. They do not disable triggers that match or fire in a less-privileged mode, and they do not affect ACTION=1. Delegated ACTION=0 triggers follow **Sdtrig**'s native-trigger match and reentrancy rules. Because **tcontrol** is not accessible from S-mode, it does not provide a less-privileged analogue of the M-mode handler-reentrancy control.

6.6. Debugger Access from U-mode and VU-mode

When Smuedbgsec is implemented, `mdbggen=0`, `SEDBGGEN=0`, and `UEDBGEN=1`, an external debugger whose [debug access privilege](#) is U-mode can access triggers through the `utselect + uiselect/uiereg*` interface, provided that the applicable TR bits permit access (`mstateen0.TR=1` and, when S-mode is implemented, `sstateen0.TR=1`).

When Smuedbgsec is implemented, `mdbggen=0`, `SEDBGGEN=0`, `VSEDBGGEN=0`, and `VUEDBGEN=1`, an external debugger whose debug access privilege is VU-mode uses the same interface, provided that `mstateen0.TR=1`, `hstateen0.TR=1`, and virtual `sstateen0.TR=1`.



This access path is one of the primary motivations for Sutcfg: without it, an external debugger restricted to U-mode or VU-mode privilege has no path to trigger state.

6.7. Trigger Allocation

When the applicable TR bits permit access, U-mode and VU-mode can access delegated trigger state through `utselect` and `uiselect/uiereg*` (subject to DMODE restrictions and type-specific constraints). When S-mode is implemented and `mstateen0.TR=1`, S/HS-mode can use the same access path as a manager for U-mode trigger context.

DMODE takes priority over delegated VS-mode, U-mode, and VU-mode access. When `tdata1.DMODE=1`:

- Ordinary VS-mode, U-mode, and VU-mode software can write the trigger's `tdata*` fields only under the base Sdtrig rules. When `mdbggen=0`, M-mode writes to `tdata1.DMODE` follow [\[mdbggen-dmode\]](#). S/HS-mode manager accesses to U-mode trigger state are the exception defined in [Section 6.3.2](#) and may manage `DMODE=1` state.
- VS-mode, U-mode, and VU-mode accesses through `uiereg*` follow [Section 6.3.2](#): they are read-only 0 if the hart is not in Debug Mode, and are not forced to read-only 0 if the hart is in Debug Mode. S/HS-mode manager accesses through the same aliases are not forced to read-only 0.

Higher-privilege software can reconfigure triggers with `DMODE=0` through its own interface, overriding lower-privilege configuration. A `DMODE=1` trigger is not reclaimed by VS-mode, U-mode, or VU-mode software through the delegated interface while the hart is not in Debug Mode. S/HS-mode manager access can park and restore `DMODE=1` trigger state assigned to U-mode contexts through `uiereg*`; HS-mode manages VS/VU state through [Section 4.4.3](#). Debug Mode can reclaim a trigger through the corresponding delegated interface. When `mdbggen=0`, M-mode can park or restore native trigger state as specified in [\[mdbggen-dmode\]](#).

Software must manage trigger allocation cooperatively. TR enables delegated trigger access but does not identify per-trigger ownership. There is no per-trigger hardware allocation. Before setting the applicable TR bit, higher-privilege software should clear its own mode-enable bits on any trigger it is exposing.

- M-mode allocates triggers between its own use and lower-privilege delegation.
- When S-mode is implemented, S/HS-mode software allocates triggers between its own use and U-mode delegation, and sets `sstateen0.TR=1` once it is prepared to delegate trigger access to U-mode.
- When S-mode is not implemented, M-mode firmware sets `mstateen0.TR=1` once it is prepared to delegate trigger access to U-mode.

6.8. Dependencies

Sutcfg depends on the following extensions:

- Sdtrig — Base trigger extension defining trigger CSRs and types.
- [Sucsrind](#) — User-mode indirect CSR interface (**uiselect/uireg***).
- [Smtdeleg](#) and [Sstcfg](#) (required when S-mode is implemented) — Define S-mode trigger access through **stselect** and **siselect/sireg***.
- Smstateen / Ssstateen — State enable framework for access control [6].
- H extension (optional) — Required for trigger delegation controls in VS/VU-mode.

6.9. Summary of New State

Table 22. New CSRs Introduced by Sutcfg

CSR	Address	Privilege	Notes
utselect	address TBD	URW	User trigger select

Table 23. New Smstateen Bits Introduced by Sutcfg

Register	Bit	Purpose
ssstateen0	3 (TR)	Controls U-mode delegated trigger access



mstateen0.TR and **hstateen0.TR** are defined by [Smtdeleg](#). In implementations without S-mode, **mstateen0.TR** is defined directly by *Sutcfg*.

Chapter 7. Debug Module Security (non-ISA) Extension

This chapter defines the required security enhancements for the Debug Module. The debug operations listed below are modified by this extension.

- Halt
- Reset
- Keepalive
- Abstract commands (Access Register, Quick Access, Access Memory)
- System bus access

If any hart in the system implements Smmedbgsec, the Debug Module must also implement the Debug Module Security Extension.

7.1. External Debug Security Extensions Discovery

The ISA and non-ISA external debug security extensions impose security constraints and introduce non-backward-compatible changes. The presence of the extensions can be determined by polling the ALLSECURED and/or ANYSECURED bits in **dmstatus** Table 24. These bits will read as 0 when **psecdbggen** is 0, regardless of whether the harts implement Smmedbgsec. When **psecdbggen** is 1, ALLSECURED is set to 1 if all currently selected harts implement Smmedbgsec, and ANYSECURED is set to 1 if any currently selected hart implements Smmedbgsec. When any hart implements Smmedbgsec, it indicates that the Debug Module implements the Debug Module Security Extension as described in this chapter.

7.2. Halt

The halt behavior for a hart is detailed in Section 3.1. According to *The RISC-V Debug Specification* [1], a halt request must be responded to within one second. However, this constraint must be relaxed, as the request might remain pending when debugging is disallowed. In the case of a halt-on-reset request (SETRESETHALTREQ), the request is only acknowledged by the hart once it has reached a privilege level in which debug is allowed.

7.3. Reset

The HARTRESET operation resets selected harts. When M-mode debugging is disallowed, the hart will raise a security fault error to the Debug Module on HARTRESET operations. The error can be detected through ALLSECFAULT and ANYSECFAULT in **dmstatus**.

The NDMRESET operation is a system-level reset not tied to hart privilege levels and resets the entire system (excluding the Debug Module). The Debug Module Security Extension makes the NDMRESET field read-only 0 when **psecdbggen** is 1. The debugger can determine support for the NDMRESET operation by setting the field to 1 and subsequently verifying the returned value upon reading.

7.4. Keepalive

The SETKEEPALIVE bit serves as an optional request for the hart to remain available for debugging. This bit only takes effect when M-mode debugging is allowed; otherwise, the hart behaves as if the bit is not set.



*Keepalive affects whole-hart power and availability, including M-mode. Halt and single-step requests with **mdbgen**=0 remain privilege-scoped and do not imply hart-global power control.*

*Therefore, a lower-privilege debug enable does not grant the debugger authority to keep the hart available through keepalive, and **SETKEEPALIVE** remains read-only 0 when **mdbgen**=0. If keepalive is needed later, it should be introduced as separate M-mode delegation extension rather than implied by **mdtcfg.*EN**.*

7.5. Abstract Commands

The hart's response to abstract commands is detailed in [Section 3.1](#). The following subsection delineates the constraints when the Debug Module issues an abstract command.

7.5.1. Relaxed Permission Check **reLaxedpriv**

The **reLaxedpriv** field is hardwired to 0.

7.5.2. Address Translation **AAMVIRTUAL**

The field **command.AAMVIRTUAL** determines whether the Access Memory command uses a physical or virtual address. When **mdbgen**=1, the behavior follows the original RISC-V Debug Specification [1]. When **mdbgen**=0:

- If **AAMVIRTUAL**=0, the hart responds with a security fault error (setting **abstractcs.CMDERR** to 6).
- If **AAMVIRTUAL**=1, addresses are treated as virtual and are translated and protected according to the debug access privilege, as defined in [Section 3.1.3](#), and may be modified to the effective debug access privilege as defined in [Section 3.1.6.2](#) or [Section 3.1.7.2](#).

7.5.3. Quick Access

When M-mode debugging is disallowed (**mdbgen**=0) for a hart, any Quick Access operation will be discarded by the Debug Module, causing **abstractcs.CMDERR** to be set to 6.



*Quick Access abstract commands initiate a halt, execute the Program Buffer, and resume the selected hart. These halts cannot remain pending until debugging is allowed because the debugger blocks while waiting for completion. To address this, the hart would need to distinguish between Quick Access and other halt requests. Since Quick Access is an optional optimization, the Debug Module Security Extension avoids this additional hardware complexity by disallowing Quick Access when **mdbgen** is 0.*

7.6. System Bus Access

The System Bus Access must be checked by bus initiator protection mechanisms such as IOPMP [7] or RISC-V Worlds [8]. The bus protection unit can return an error to the Debug Module on illegal accesses; in that case, the Debug Module will set **sbcS.SBERROR** to 6 (security fault error).



Trusted entities like the RoT should configure IOPMP or equivalent protection before external debug is allowed.

7.7. Security Fault Error Reporting

A dedicated error code, security fault error (CMDERR 6), is included in **abstractcs.CMDERR**. Issuance of abstract commands under disallowed circumstances sets **abstractcs.CMDERR** to 6. Additionally, the bus

security fault error (SBERROR 6) is introduced in **sbc**s.SBERROR to denote errors related to system bus access.

Error status bits are internally maintained for each hart. The ALLSECFAULT and ANYSECFAULT fields in **dmstatus** indicate the error status of the currently selected harts. These error statuses are sticky and can only be cleared by writing 1 to **dmcs2.ACKSECFAULT** Table 25.

7.8. Platform Debug Security Enable

The platform state element **psecdbgen** is introduced to enable the debug security constraints defined by this specification. When **psecdbgen** is 0, the platform retains full debugging capabilities, as if the Secure Debug extensions were not implemented:

- All [debug operations](#) are executed with M-mode privilege (equivalent to having **mdbg** set to 1) for all harts in the system.
- The NDMRESET operation is allowed.
- The RELAXEDPRIV field may be configurable.
- System Bus Access may bypass the bus initiator protections.

When **psecdbgen** is 1, the debug security constraints in this specification apply.



*If **mdtcf**g is implemented, it remains accessible to M-mode software when **psecdbgen** is 0. Implemented debug-related fields retain their specified WARL behavior, but their values do not constrain external debug; the unrestricted debug behavior defined above applies. The trace-related fields remain independent of **psecdbgen** and retain their specified effect.*

7.9. Authorization Handoff

An **authorization handoff** occurs when the platform ends debug for one principal or domain, or admits a different one. That includes one S-mode domain finishing debug before another starts.

A hart entering or leaving a debug-allowed privilege mode is not a handoff, as long as the same principal or domain still holds debug authority. Changing **psecdbgen** or a hart's [debug access privilege](#) is not a handoff by itself.

After a handoff, the newly admitted debugger must not observe or use Debug Module state obtained under authorization it does not have. An operation issued before the handoff must not complete afterwards in a way that makes such state observable or usable.

The platform must complete sanitization before the succeeding debugger is admitted. The mechanism is implementation-specific.



The following state needs an explicit hardware or software clear/invalidate path across a handoff:

- Must sanitize: **data***; **progbuf*** if it is DMI-readable; **sbdata***.
- Should also review: **sbaddress*** (last SBA address); **command** (e.g. **regno**); **abstractauto**; and the SBA auto bits in **sbc**s (**sbreadondata**, **sbreadonaddr**, **sbautoincrement**). Leftover control can re-issue an earlier access or reveal what was touched.

Clearing existing register contents is insufficient if an earlier operation can later repopulate them.

One option is to set **dmcontrol.DMACTIVE** to 0 and wait for the state change to complete, which resets Debug Module state. **NDMRESET** does not reset the Debug Module. Software sanitization is another option. Writing zero to the affected registers is not a generic sanitization procedure: when **abstractauto** is set, accessing **data** or **progbuf** may execute command, and writing **sbdata0** starts a system bus write.

7.10. Update of Debug Module Registers

The Debug Module Security Extension introduces new fields in the Debug Module registers. In **dmstatus**, **ANYSECURED** and **ALLSECURED** are added at bit 20 and bit 21 respectively, while **ANYSECFAULT** and **ALLSECFAULT** are added at bit 25 and bit 26. The **ACKSECFAULT** field is added to **dmcs2** at bit 12.

Table 24. Details of newly introduced fields in **dmstatus**

Field	Description	Access	Reset
ALLSECURED	The field is 0 when psecdbggen is 0. When psecdbggen is 1, this field is 1 if all currently selected harts implement Smmdbgsec .	R	-
ANYSECURED	The field is 0 when psecdbggen is 0. When psecdbggen is 1, this field is 1 if any currently selected hart implements Smmdbgsec .	R	-
ALLSECFAULT	The field is 1 when all currently selected harts have raised a security fault due to reset operation	R	-
ANYSECFAULT	The field is 1 when any currently selected hart has raised a security fault due to reset operation	R	-

Table 25. Detail of **ACKSECFAULT** in **dmcs2**

Field	Description	Access	Reset
ACKSECFAULT	0 (nop): No effect. 1 (ack): Clears error status bits for any selected harts.	W1	-

Appendix A: Valid Extension Combinations

This appendix lists valid external debug and trace extension combinations for the supported privilege-mode architectures. The two families are independent, so each table applies only to the family it covers.

A.1. External Debug Security Extension Combinations

Table [Table 26](#) shows valid external debug extension combinations for different architectures:

Table 26. Valid External Debug Extension Combinations

Architecture	Valid Debug Extension Combinations
M-only	• Smmedbgsec
M/U	• Smmedbgsec • Smmedbgsec + Smuedbgsec
M/S/U	• Smmedbgsec • Smmedbgsec + Smsedbgsec • Smmedbgsec + Smsedbgsec + Smuedbgsec
M/S/VS/VU/U	• Smmedbgsec • Smmedbgsec + Smsedbgsec • Smmedbgsec + Smsedbgsec + Smuedbgsec • Smmedbgsec + Smsedbgsec + Smvsgedbgsec • Smmedbgsec + Smsedbgsec + Smvsgedbgsec + Smuedbgsec

Smtdeleg, Sstcfg, Sucsrind, and Sutcfg are distinct extensions specified in [Chapter 4](#), [Chapter 5](#), and [Chapter 6](#). Sstcfg requires Smtdeleg. Sutcfg requires Sucsrind and, when S-mode is implemented, also requires Smtdeleg and Sstcfg. When implemented, these extensions provide S/HS-mode, VS-mode, and U/VU-mode access to Sdtrig trigger state for a lower-privilege debugger; none is a prerequisite for the combinations listed above.

A.2. Trace Security Extension Combinations

Table [Table 27](#) shows valid trace extension combinations for different architectures:

Table 27. Valid Trace Extension Combinations

Architecture	Valid Trace Extension Combinations
M-only	• Smmetricsec
M/U	• Smmetricsec • Smmetricsec + Smuetrcsec
M/S/U	• Smmetricsec • Smmetricsec + Smsetrcsec • Smmetricsec + Smsetrcsec + Smuetrcsec
M/S/VS/VU/U	• Smmetricsec • Smmetricsec + Smsetrcsec • Smmetricsec + Smsetrcsec + Smuetrcsec • Smmetricsec + Smsetrcsec + Smvsetrcsec • Smmetricsec + Smsetrcsec + Smvsetrcsec + Smuetrcsec

Appendix B: Software Debug/Trace Security Policy Management

This section provides guidance on how software at different privilege levels can manage debug/trace security policies.

B.1. M-mode Management

M-mode software is responsible for managing the `SEDBGGEN`, `VSEDBGGEN`, `UEDBGGEN`, and `VUEDBGGEN` fields in the `mdtcfg` CSR for external debug, and the `SETRCEN`, `VSETRCEN`, `UETRCEN`, and `VUETRCEN` fields in the `mdtcfg` CSR for trace. The `mdbggen` and `mtrcen` states are read-only to software and are typically controlled by platform hardware, such as an RoT or lifecycle fuses.

M-mode software should:

1. Initialize debug/trace security policy for lower privilege modes based on security requirements.
2. Provide interfaces for dynamic debug/trace security policy updates (e.g., through SBI calls from lower privilege modes).
3. Save and restore debug/trace security policies when switching between supervisor software contexts.
4. When `mdbggen`=0, save and restore trigger context at lower-privilege software boundaries using the M-mode `DMODE` access in [\[mdbggen-dmode\]](#). Read and save `tselect` and `tdata*` for each trigger. To park a `DMODE`=1 trigger, write `tdata1` with a disabled-trigger encoding (`TYPE`=15 where supported, `DMODE`=0, and `ACTION`=0), so the trigger cannot match while its state is being changed; if the write would otherwise leave `ACTION`=1, the implementation sets `ACTION` to 0. Clear its mode-enable bits before the next lower-privilege context runs. To restore a saved configuration, first write the disabled-trigger encoding, then write `tdata2` and `tdata3`, and finally write the saved `tdata1`, including `DMODE`=1 and `ACTION`=1 in a single `tdata1` write if that was the saved configuration.
5. When `Smtdeleg` is implemented, set `mstateen0.TR` when prepared to delegate trigger access to S/HS-mode.
6. Save and restore `tselect` around delegated trigger accesses because `stselect`, `utselect`, and `tselect` share the same selection state.
7. When `Sutcfg` is implemented without S-mode, set `mstateen0.TR` when prepared to delegate trigger access to U-mode.

B.2. S/HS-mode Management

S/HS-mode software is responsible for managing the debug/trace security policy for lower privilege modes:

1. Call M-mode to apply the debug/trace security policy before launching an application or VM.
2. Provide interfaces to handle dynamic debug/trace security policy update requests (e.g., SBI calls from VMs) and call M-mode to apply the security policy accordingly.
3. Save and restore debug/trace security policies when switching between applications or VMs.
4. When `Sstcfg` is implemented and `mstateen0.TR`=1, save and restore the visible delegated S/HS-mode trigger context (typically `DMODE`=0) through `stselect` and `siselect/sireg*` when switching supervisor contexts. For each visible S/HS-mode trigger, save and restore `stselect` and `siselect`, write `sireg` with the disabled-trigger encoding (`TYPE`=15, `DMODE`=0, and `ACTION`=0) before writing `sireg2` or `sireg3`, and then write the final `sireg` value; `sireg5` and `sireg6` are illegal for this access. When HS-mode manages VS-mode or VU-mode contexts, save and restore their trigger context through `stselect` and

vsiselect/vsireg*, including **DMODE=1** state, as specified in [Section 4.4.3](#).

5. When **Sstcfg** is implemented and the hart is not in Debug Mode, skip a trigger index if **sireg** through **sireg4** all read as zero. The slot is reserved by Debug Mode (**DMODE=1**) or is otherwise unavailable. An S/HS-mode external debugger programs **DMODE=1** and **ACTION=1** through the same interface from Debug Mode; see [Section 4.4.2](#). A **DMODE=1** slot in the S/HS view is not reclaimed through **sireg***; request M-mode assistance as in [\[mdbgen-dmode\]](#).
6. When **Sutcfg** is implemented, set **sstateen0.TR** when prepared to delegate trigger access to U-mode, and save and restore U-mode trigger context through **utselect** and **uiselect/uireg***, including **DMODE=1** state, because **utselect** shares the same selection state.

B.3. VS-mode Management

VS-mode software is responsible for managing the debug/trace security policy for applications in VU-mode:

1. Call S/HS-mode to apply the debug/trace security policy before launching an application.
2. Save and restore debug/trace security policies when switching between applications.
3. When **Sstcfg** is implemented and the hart is not in Debug Mode, skip a trigger index if **vsireg** through **vsireg4** all read as zero. The slot is reserved by Debug Mode (**DMODE=1**) or is otherwise unavailable. A VS-mode external debugger programs **DMODE=1** and **ACTION=1** through **stselect** with **vsiselect/vsireg*** from Debug Mode; see [Section 4.4.2](#). A **DMODE=1** slot is not reclaimed by VS-mode through **vsireg***; HS-mode can manage VS/VU trigger state through direct **vsireg*** accesses as specified in [Section 4.4.3](#).
4. When **Sutcfg** is implemented and **hstateen0.TR=1**, set virtual **sstateen0.TR** when prepared to delegate trigger access to VU-mode, and save and restore **stselect** around delegated VU-mode accesses.

B.4. U-mode and VU-mode Management

When **Sutcfg** is implemented and the applicable TR bits permit access, VS-mode, U-mode, and VU-mode software can access delegated trigger state through **utselect** and **uiselect/uireg***. VS-mode, U-mode, and VU-mode software should save and restore visible trigger context (typically **DMODE=0**) when switching application contexts that use delegated triggers. When the hart is not in Debug Mode, skip a trigger index if **uireg** through **uireg4** all read as zero; the slot is reserved by Debug Mode (**DMODE=1**) or is otherwise unavailable. A U-mode or VU-mode external debugger programs **DMODE=1** and **ACTION=1** through the same interface from Debug Mode; see [Section 6.3.2](#). A **DMODE=1** slot is not reclaimed by VS-mode, U-mode, or VU-mode software through **uireg***; S/HS-mode can manage U-mode trigger state through **uireg***, and HS-mode can manage VU-mode trigger state through **vsireg***, as specified in [Section 4.4.3](#).

Appendix C: Execution-Based Implementation with Sdsec

In an execution-based implementation, the code executing the "park loop" can always run with M-mode privilege to access memory and CSRs. However, once execution is dispatched to an Abstract Command or the Program Buffer, the privilege level used to access memory and CSRs should be restricted to the [debug access privilege](#).

To achieve this, a Debug-Mode-only state element (e.g., a field in a custom CSR) may be introduced to control the privilege level in Debug Mode. When the state is set to 1, Debug Mode allows M-mode privilege; when cleared to 0, it enforces the [debug access privilege](#). The hardware sets this state to 1 upon entering the park loop and clears it to 0 with the final instruction of the park loop, immediately before execution is transferred to an Abstract Command or the Program Buffer.

Bibliography

- [1] “RISC-V Debug Specification.” [Online]. Available: github.com/riscv/riscv-debug-spec.
- [2] “RISC-V Efficient Trace for RISC-V.” [Online]. Available: github.com/riscv-non-isa/riscv-trace-spec.
- [3] “RISC-V N-Trace (Nexus-based Trace) Specification.” [Online]. Available: github.com/riscv-non-isa/riscv-nexus-trace.
- [4] “RISC-V Hart-Trace Interface (RHTI).” [Online]. Available: github.com/riscv-non-isa/riscv-hart-trace-interface.
- [5] “Smcsrind/Sscsrind Indirect CSR Access, Version 1.0.” [Online]. Available: docs.riscv.org/reference/isa/priv/indirect-csr.html.
- [6] “Smstateen/Ssstateen Extensions, Version 1.0.” [Online]. Available: docs.riscv.org/reference/isa/priv/smstateen.html.
- [7] “RISC-V IOPMP Architecture Specification.” [Online]. Available: github.com/riscv-non-isa/riscv-iopmp.
- [8] “RISC-V Worlds.” [Online]. Available: github.com/riscv/riscv-worlds.